

Module-2

Micro architecture

Day-1 — Module-1

Day-2 — Module-2

Day-3 — Module-3

Day-4 — Module-4

Microarchitecture (Note)

- Internal organization of this hardware is called micro-architecture
- micro architecture is the internal design of the computer to implement architecture
- micro architecture explains how the hardware is organized to make architectural rules to work.

Micro architecture has two parts

- ① Data path
- ② Control unit

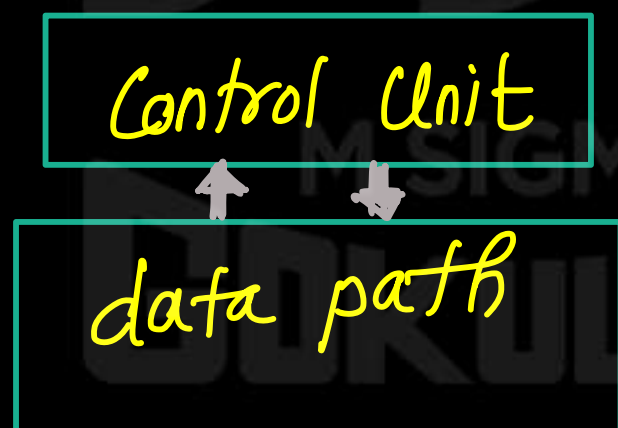


① Data path

- It is the hardware path way through which data flows inside the CPU
- Data path is the part of processor that contains the hardware used to store, move, and process the data during the execution of instructions.
- It contains structures such as memories, registers, ALU, multiplexers.

② Control unit

- Control unit receives the current instruction from the data path and tells the data-path how to execute the instruction
- Control unit produce different signal to control the operation of data path



State Elements in the data. (Note)

→ state elements are the hardware components to store and remember the information

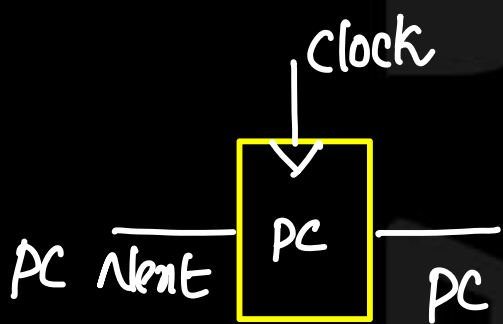
→ Different state elements are

- ① program counter (PC)
- ② Instruction memory (IM)
- ③ Register files
- ④ Data Memory.

Program Counter (PC):

— it is a register that store the address of next instruction to be executed.

→ In RISC processor instruction size is 4Byte (32bit)



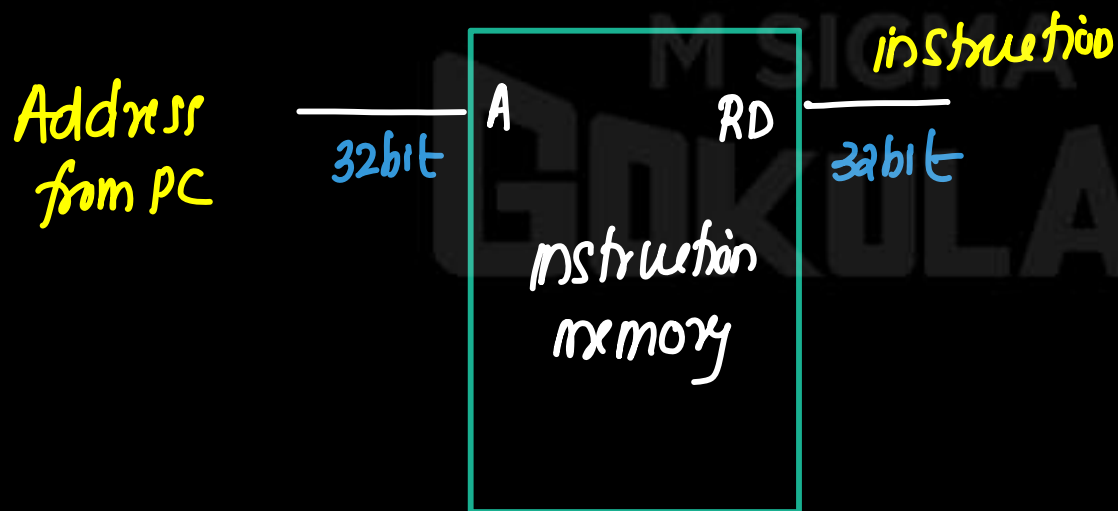
- PC Next → input to the program counter, indicates the address of next instruction to be executed
- PC → indicate the address of current instruction
- Address is 32 bit.
- clock is required to tell the pc when to update its value
- pc update its content at the edge of clock.



② Instruction Memory.

(N)

- this memory is used to store the instruction to be executed
- program counter provides the address and instruction memory outputs the instruction stored at that address. This instruction is sent to CPU for decoding and execution.
- Instruction memory has a single read port
- Read port is an interface through which instruction memory outputs an instruction when an address is given
- Read port has input lines through which 32 bit address is given to the instruction memory.
- Read port has output lines (read data output) (RD) through which instruction memory outputs stored instruction.
- read port takes 32 bit address input (A) from PC and places the instruction on the read data output (RD)



Read data input - A
(address)

Read data output → instruction

③ Register file :

②

→ Register file is a collection of registers inside the CPU

→ two operations are possible in register file,

① Read : Read the content of Register through Read port

② Write : Write the data to the register through the write port.

→ Register files have two read port and one write port

→ Read port has

① read address input

② read data output

→ read address input takes the 5 bit address (specify the address of register read access) and register file load the content of register to register data output

read address input (A_1)

read data output (RD)

→ write port has

① write address input (A_3)

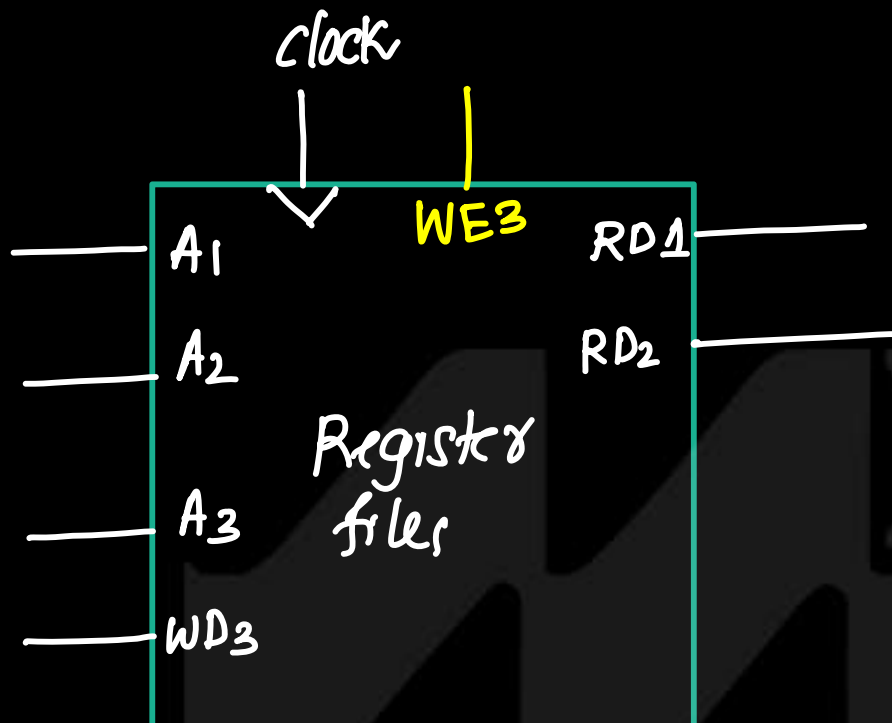
② write data (WD_3).

Register file store the data present at write data pin to the register whose address is specified in the write address inputs

→ Two operations can be performed in Registerfiles. These operations can be selected based on the values of control signal (write enable) (WE)

If $WE = 1$, perform write operation
If $WE = 0$, perform read operation

→ one clock input is required to control write operation. Data is written into the selected register only at the clock edges.



M SIGMA
GOKULAM

Data Memory

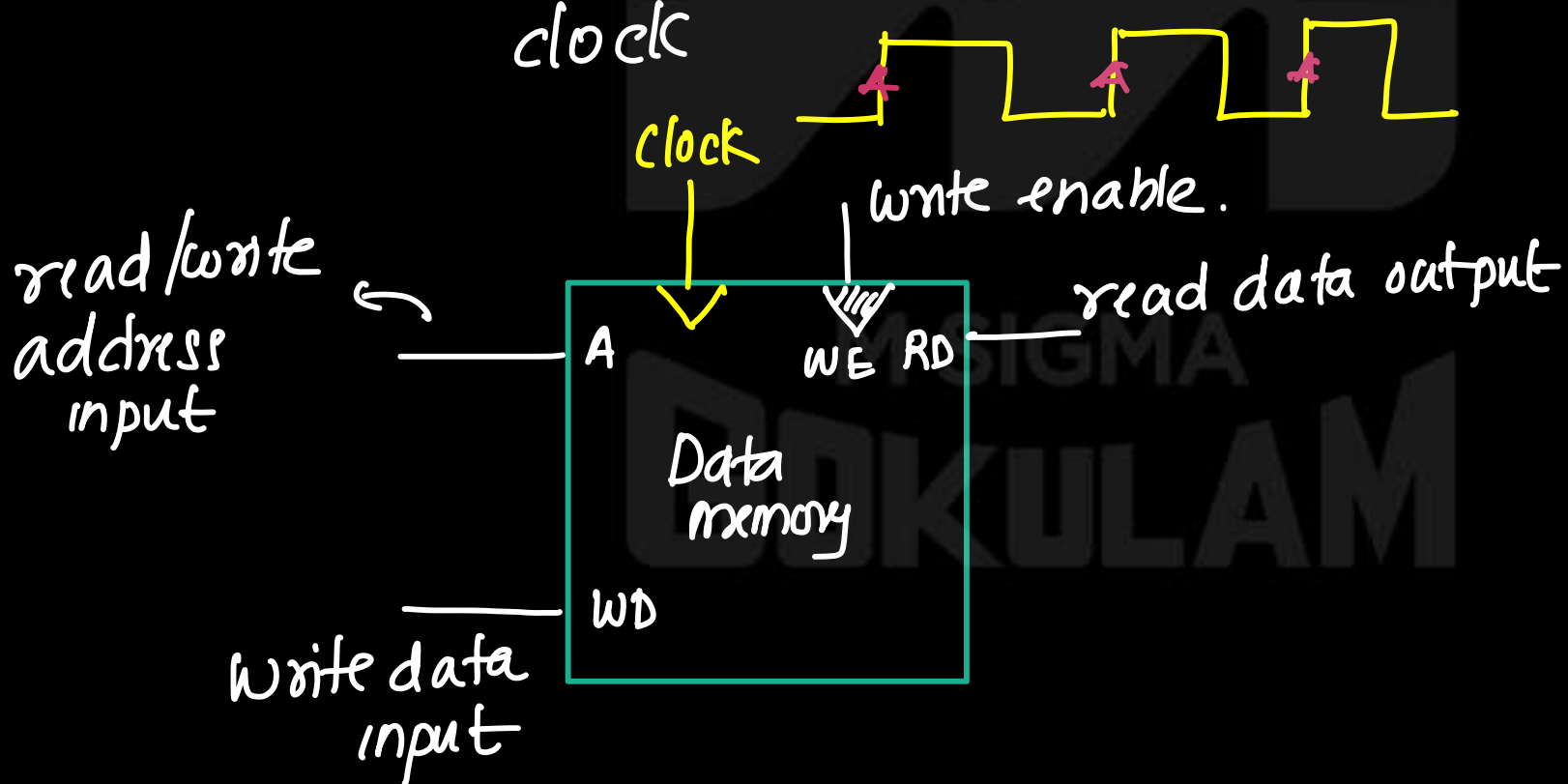
(N)

- This memory is used to store data
- This memory has single read write port
- Read write operation is selected on the value of write enable control signal

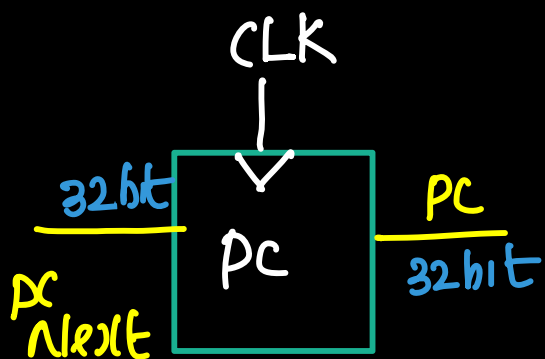
If write enable (WE) = 1, then data written on the location whose address present at the input line of read write port.

If $WE = 0$, then data is read from the memory location whose address is available at the input line of read/write port.

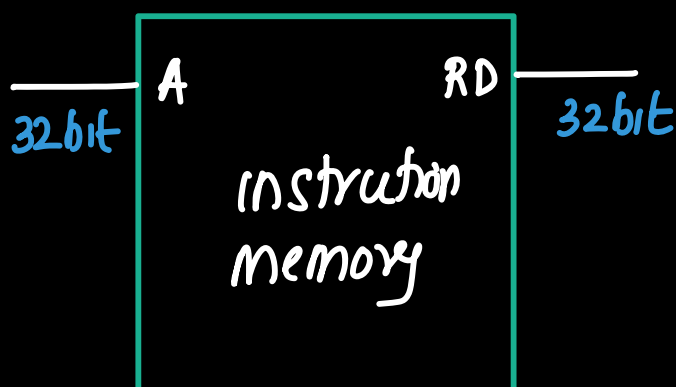
- A clock signal used to control the write operation write operation happens at the rising edge of clock



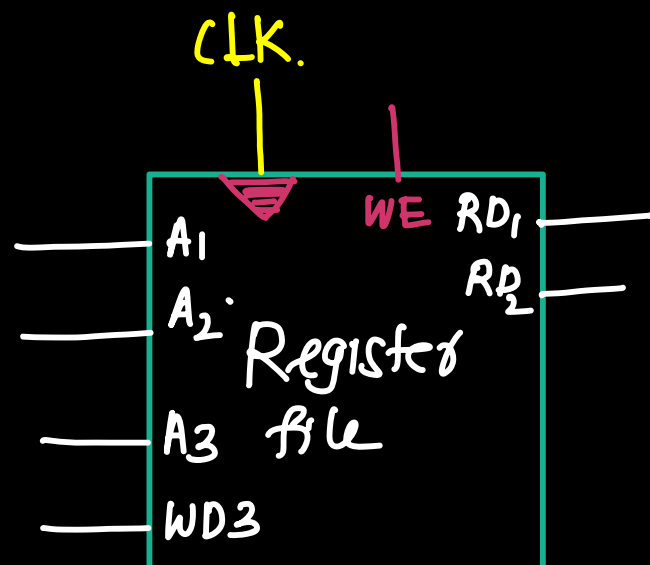
Summary



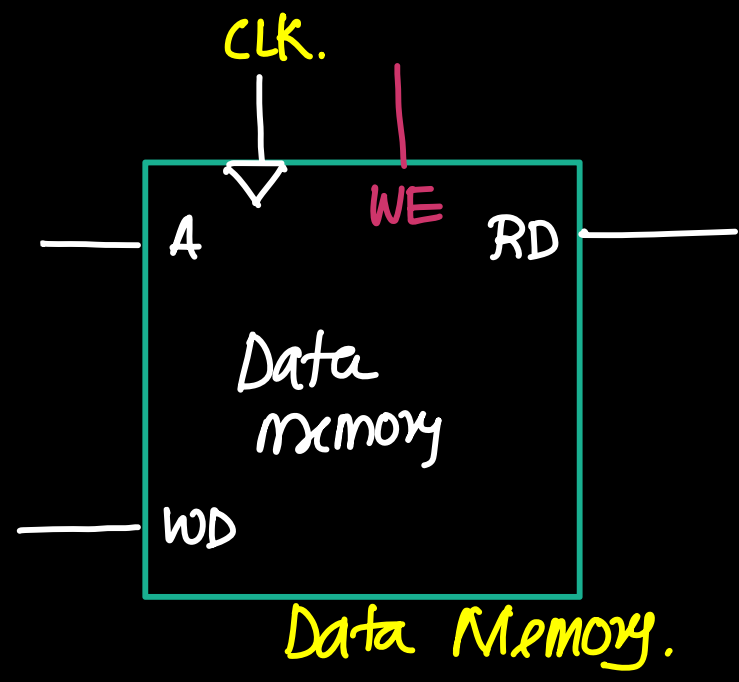
PC



Instruction memory (IM)



Register file



(N)



Commonly Used Microarchitecture (Note)

Single cycle microarchitecture:

- In this type, an instruction execute in one cycle
- Whatever may be operation, one instruction execute in one cycle
- Let consider 3 instruction I_1, I_2, I_3 and T_1, T_2, T_3

are the time required to complete operation

$$T_1 > T_2 > T_3.$$

then cycle time is limited by slowest instruction.

(clock type $\leq T_i$)

- In this architecture, processor requires separate memory to store instruction and data.

Multicycle micro architecture

- In multicycle architecture, split the instruction execution into several stages. (IF, ID, execution, memory access, write back)
- each stage complete in one clock cycle.
- In this architecture instruction execute in a series of shorter cycles.
- Multicycle processor executes only one instruction at a time, but each instruction takes multiple clock cycle.

Pipelined Microarchitecture:

(N)

- In this architecture instructions are divided into several stages (IF, ID, EXE, memory access, write back) and multiple instructions are execute at same time.
- this architecture improves the processor performance by overlapping the execution of multiple instruction
- Each instruction require multiple clock cycle to complete.



Why is it necessary to have separate memories for Instructions and Data in a single-cycle implementation, Justify your answer

(A)

- In single cycle processor instruction execution completes in one cycle
- during the execution, processor need to fetch the instruction as well as data
- If we are using a single Memory for both instruction data, then it will cause hazards.



Performance Analysis of Microarchitecture (Note)

→ Performance of microarchitecture refers to the how efficiently a processor execute the instruction. this is evaluated using execution time of a program.

→
$$\text{Execution time} = \text{no: of instructions} \times \text{CPI} \times \text{clock cycle time}$$

— no: of instruction = total no: of instructions in the program.

— CPI (clock cycle per instruction)
average no: of cycles required to execute one instruction

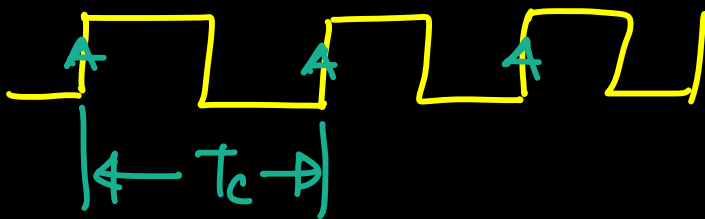
$$\text{CPI} = \frac{\text{no: of cycles}}{\text{Instruction}}$$

eg: $\text{CPI} = 1$ one cycle is required to complete one instruction.

$\text{CPI} = 4$ 4 cycle is required to complete one instruction

— clock cycle time (T_c)

time duration of one cycle.



→ $T_c = \frac{1}{f_c}$ $f_c \rightarrow$ no: of cycles in one sec

→ execution time of a program depends on

(1) no: of instruction

(2) CPI

(3) T_c

- To reduce the execution time, reduce any one factor
→ But when we increase $f_c (\frac{1}{T_c})$, that doesn't provide better performance. So we need to design a good micro architecture to reduce the execution time

— throughput

No. of instructions completed in unit time

single cycle = low throughput

Multi cycle = Moderate throughput

pipelined = High throughput.



A RISC-V based processor is running a signal processing task with 5×10^9 instructions. The processor runs at 2.0 GHz with an average CPI of 1.2. If the configuration of the processor is modified after overclocking the processor to 2.5 GHz, the increased memory latency and pipeline stalls cause the CPI to rise to 1.6. Calculate the total Execution Time for both configurations in seconds and determine if the performance

Q

Case-1

$$\text{No. of instructions} = 5 \times 10^9$$

$$f = 2 \times 10^9, \text{ clock cycle time} = \frac{1}{f} = \frac{1}{2 \times 10^9} = \underline{\underline{5 \times 10^{-10}}}$$

$$\text{CPI} = 1.2 \text{ (clock per instruction)}$$

$$\text{Execution time (Et)} = \text{no. of instructions} \times \text{CPI} \times \text{clock cycle time}$$

$$\text{Et} = 5 \times 10^9 \times 1.2 \times 5 \times 10^{-10} =$$

$$= 3$$

Case 2

$$\text{CPI} = 1.6$$

$$f = 2.5 \times 10^9, \text{ clock cycle time} = \frac{1}{f} = \frac{1}{2.5 \times 10^9} = \underline{\underline{4 \times 10^{-10}}}$$

$$\text{Et} = 5 \times 10^9 \times 1.6 \times 4 \times 10^{-10} = \underline{\underline{3.2}}$$

Case 1 $\rightarrow$ Execution time is 3s

Case 2 - Execution time is 3.2s

so performance degraded.

Single cycle processor Data path & execution (Note)

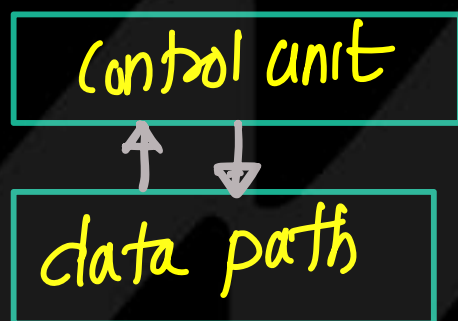
→ Single cycle processor execute one instruction in one clock cycle.

→ Designing of single cycle processor

① Constructing the data path by connecting the state elements and combinational logic circuits.

② Construct control unit using combinational logic circuit that generate appropriate control signal.

③ Analyze the performance of single cycle processor



Single cycle processor — sample program — ①

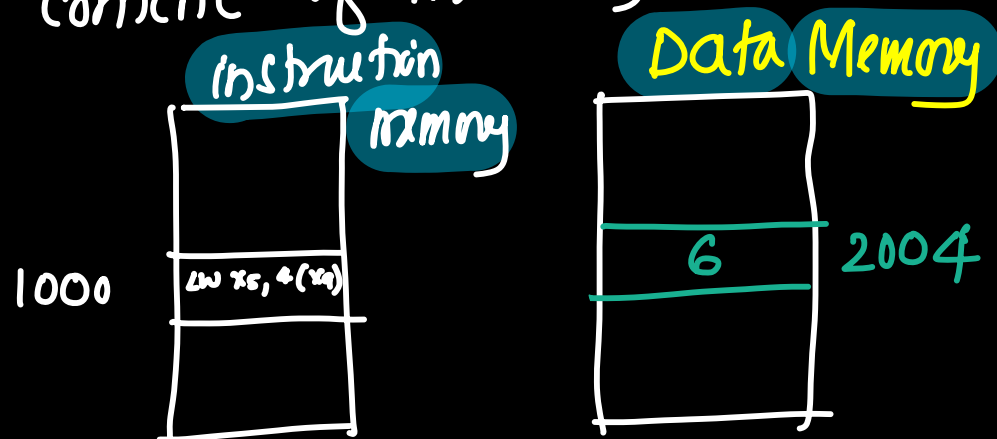
① LW $x_5, 4(x_9)$

→ x_5, x_9 are the registers in Register file

Content of $x_9 = 2000H$ →

→ Content of PC = 1000 → address of instruction stored at 1000 in the instruction memory.

→ Content of memory location 2004 = '6'



→ LW $x_5, 4(x_9)$
load the content of memory location pointed by $4(x_9)$ to the register x_5

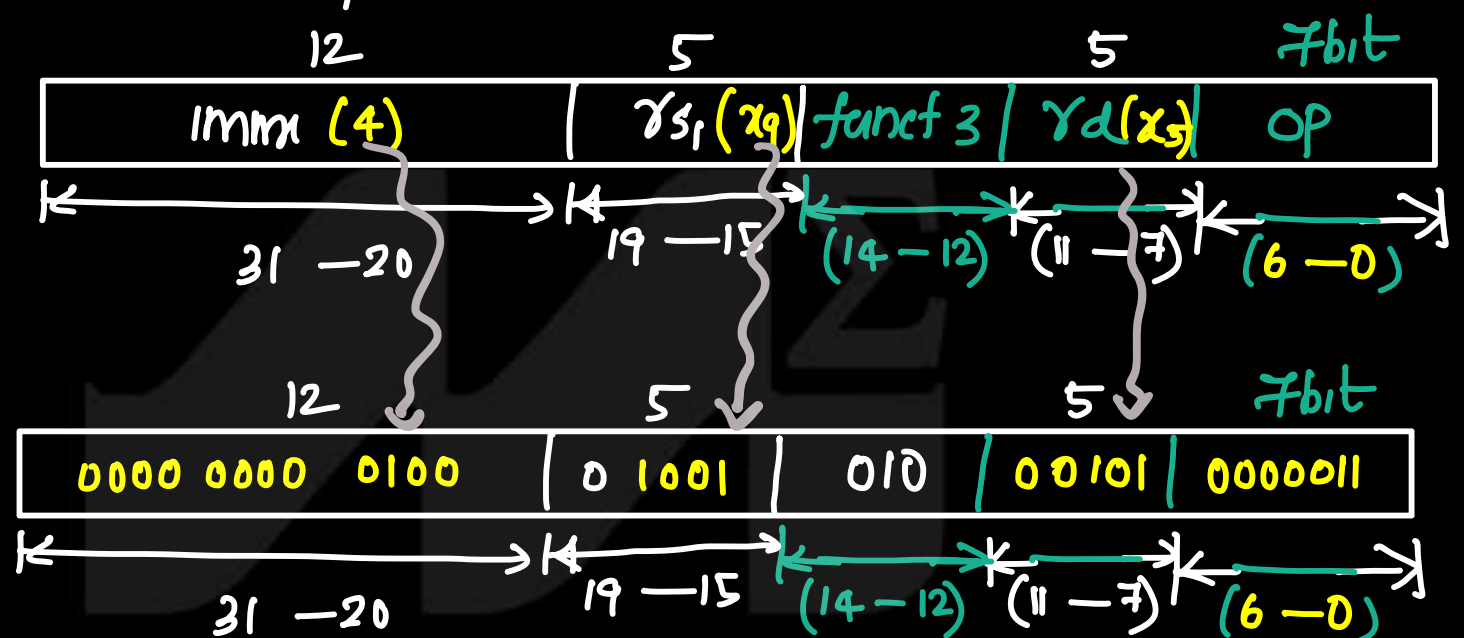
effective address of memory location can be determined by adding base address with offset address

- ① base add is present in $x_9 \Rightarrow 2000$
- ② offset = 4

$$\text{effective address} = 2000 + \frac{4}{1} = 2004$$

→ Content of 2004 is moved to x_5 .

→ $\text{LW } x_5, 4(x_9)$ is a I-type instruction



Instn 6:0 → represent opcode

Instn 11:7 → destination

Instn 14:12 → funct3 bits

Instn 19:15 → source

Instn 31:20 → represent immediate value

base address, represented by instn 19:15 bits
offset (4) represented by instn 31:20 bits.

Step 1 → Construct data path.

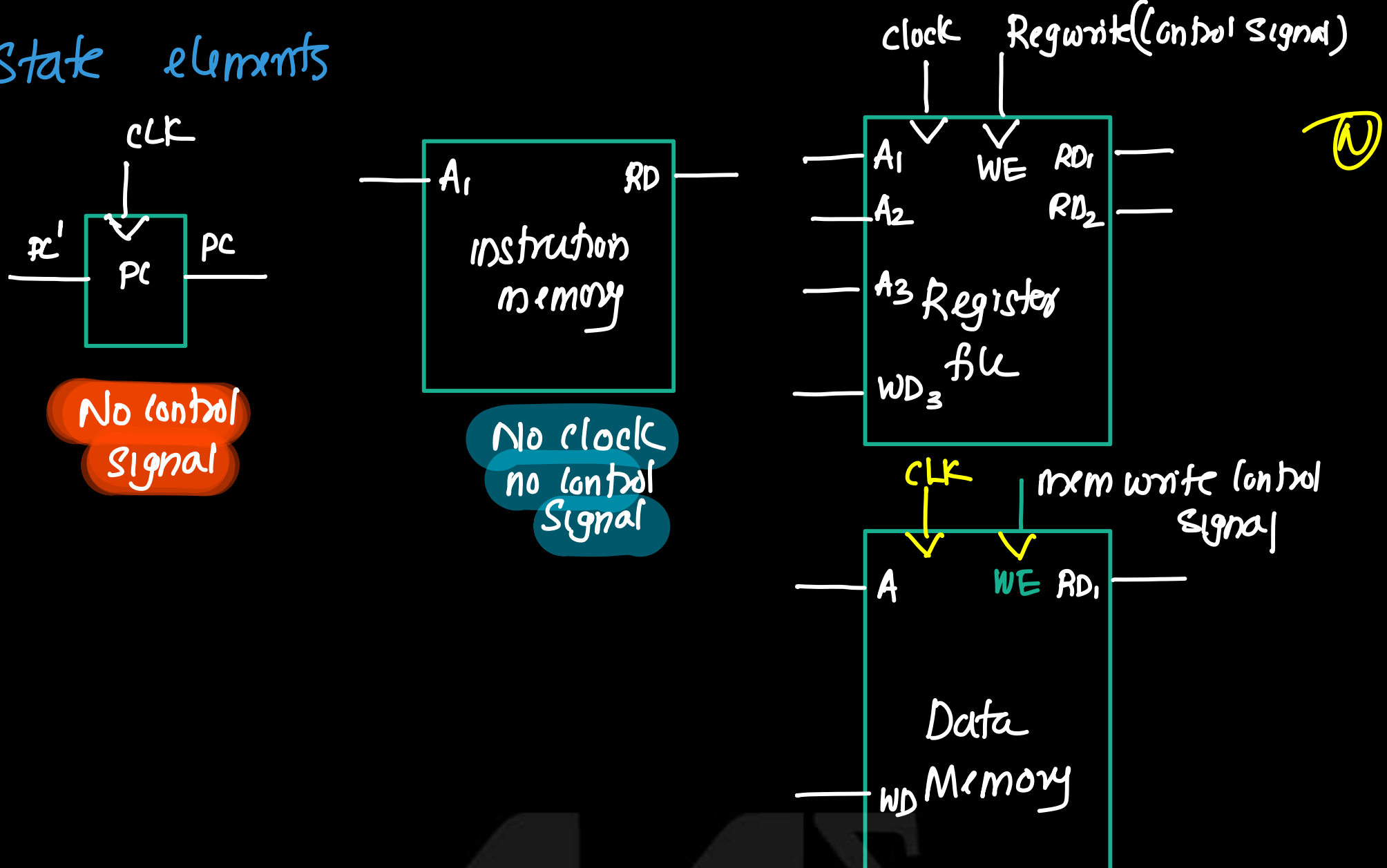
① state elements required

① pc, Instruction memory, Register file, data memory

② Additional logic circuit

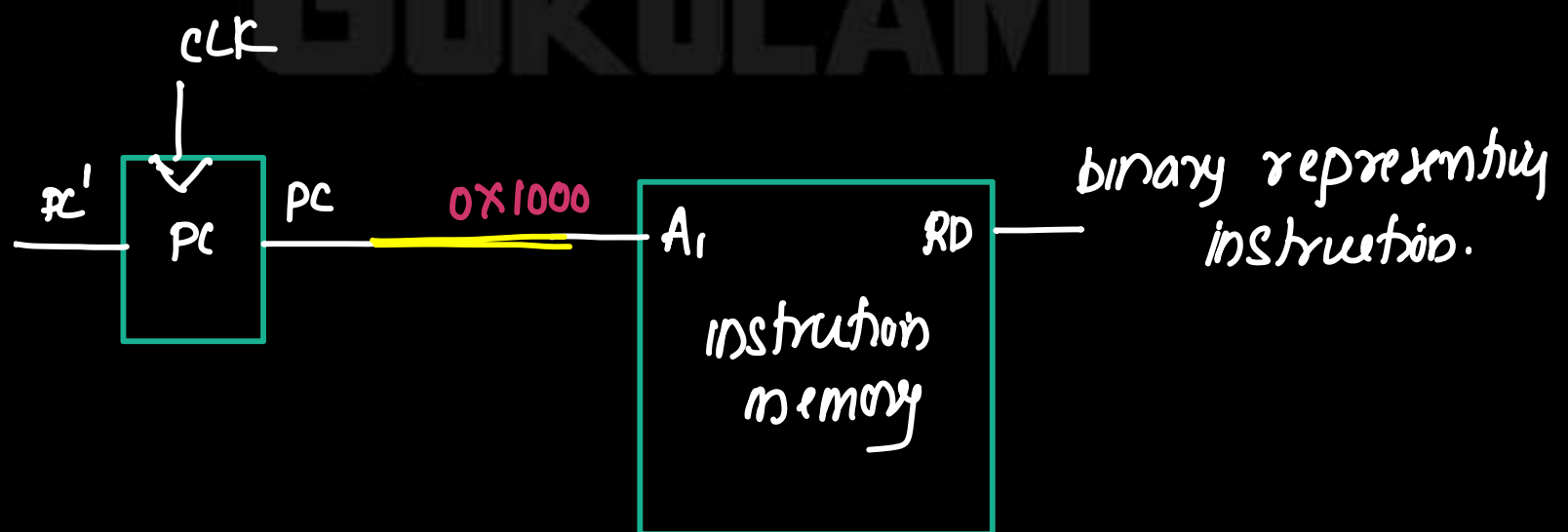
① ALU

State elements



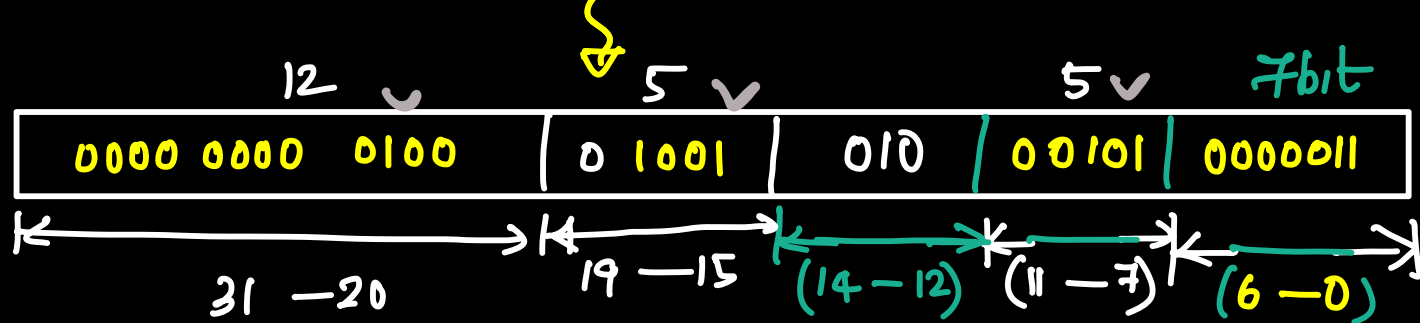
Instruction Execution Steps.

- ① PC contains address of next instruction to be executed
 $PC = 1000H$
- ② PC output is directly connected to the address input of instruction memory.



- ② instruction memory fetch the instruction whose address is present at the address input and output the instruction through read data output (RD_1). thus instruction fetch is completed.

$RD_1 \rightarrow$ contains machine code (binary) of instruction



Instn 6:0 → represent opcode

Instn 11:7 → destination

Instn 14:12 → funct 3 bits

Instn 19:15 → source

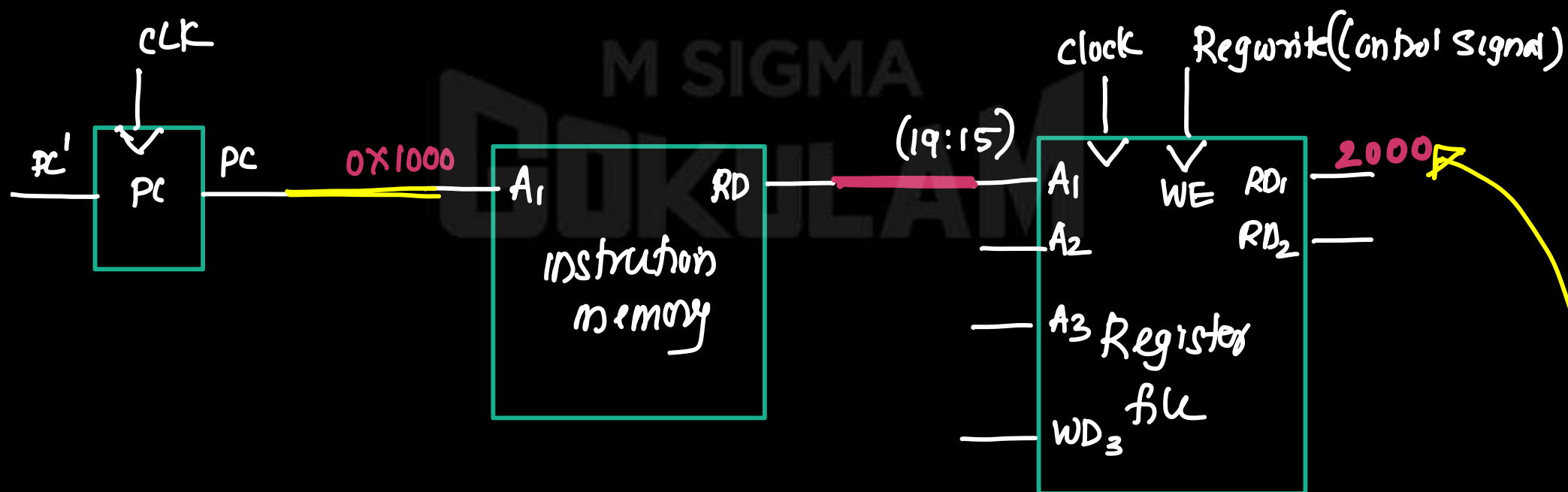
Instn 31:20 → represent immediate value

base address^(x9) represented by instn 19:15 bits
offset (4) represented by instn 31:20 bits.

③ Next is to calculate the effective address of memory location.

— read the source register(x9) containing base address.

— instruction bits 19:15 (instn 19:15) represents the source register. So connect these bits to the address input (A₁) of Register file.

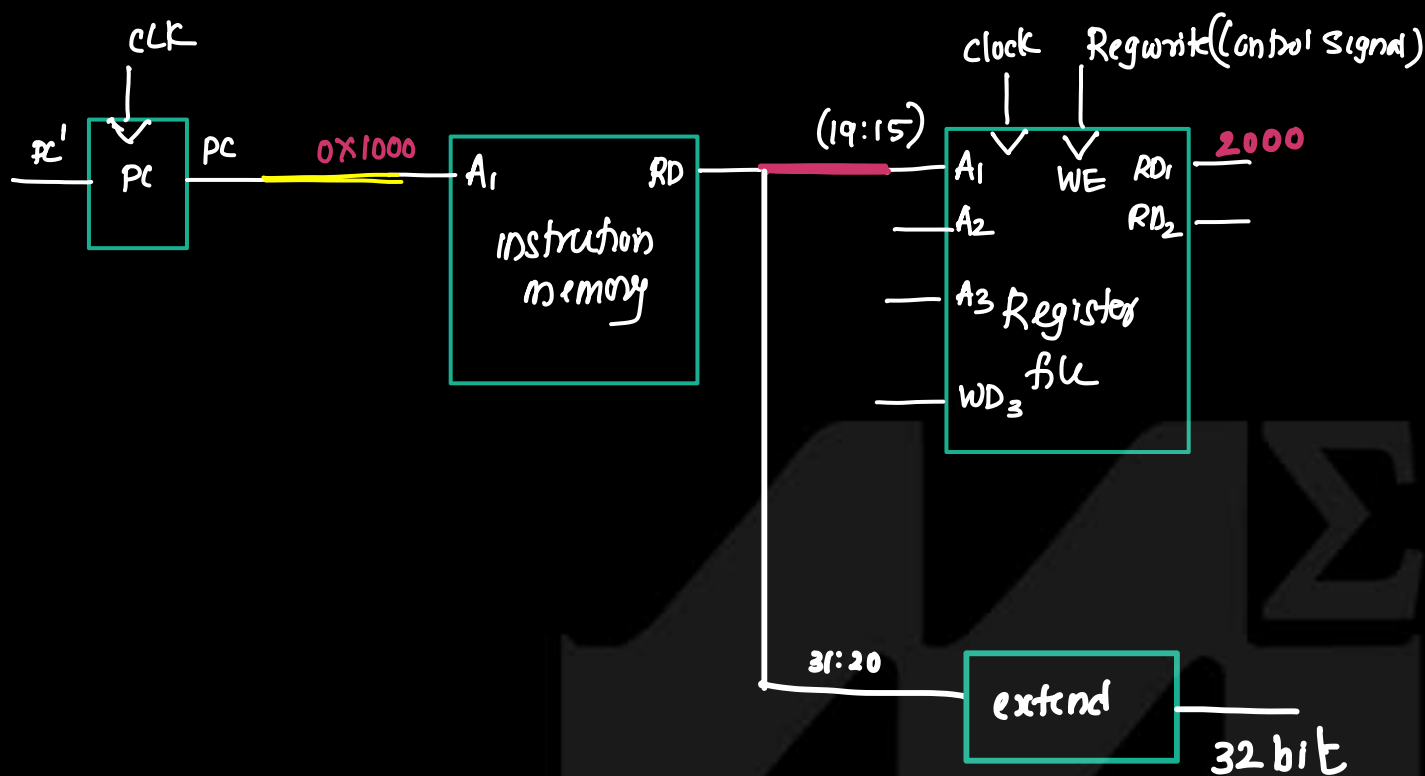


— Register file Read the value of register specified by the address in A₁ and output to the RD₁ (RD₁ = content of x₉ = 2000H)

— To calculate effective address, read offset.
offset value stored in the instruction (bits 20 to 31) (instn 31:20) → 12bit.

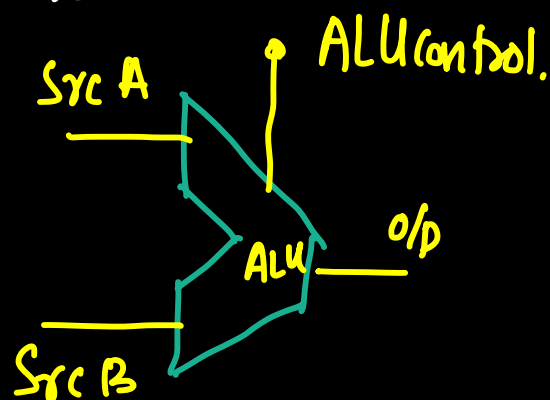
→ To calculate effective address, read to add base address (2000) and offset 4(12bit)

- to add these two extend a 12 bit (offset) to 32 bit. this can be done by passing 12 bits of instruction (instn 31:20) to the extender.



- processor adds base address (RD_1) and offset (32 bit) to get the effective address. This addition can be done by ALU.
- ALU receives two input (Src A, Src B). Src A is the base address from the register file and Src B is the offset from the 'extend'.
- ALU can perform many operations (Add, Sub etc.) So a control signal is required to specify the operation (ALU control).

If ALU Control $\rightarrow 000 \rightarrow$ then, perform addition



- ALU produce the effective address.
- output of ALU is send to the address input

of data memory.

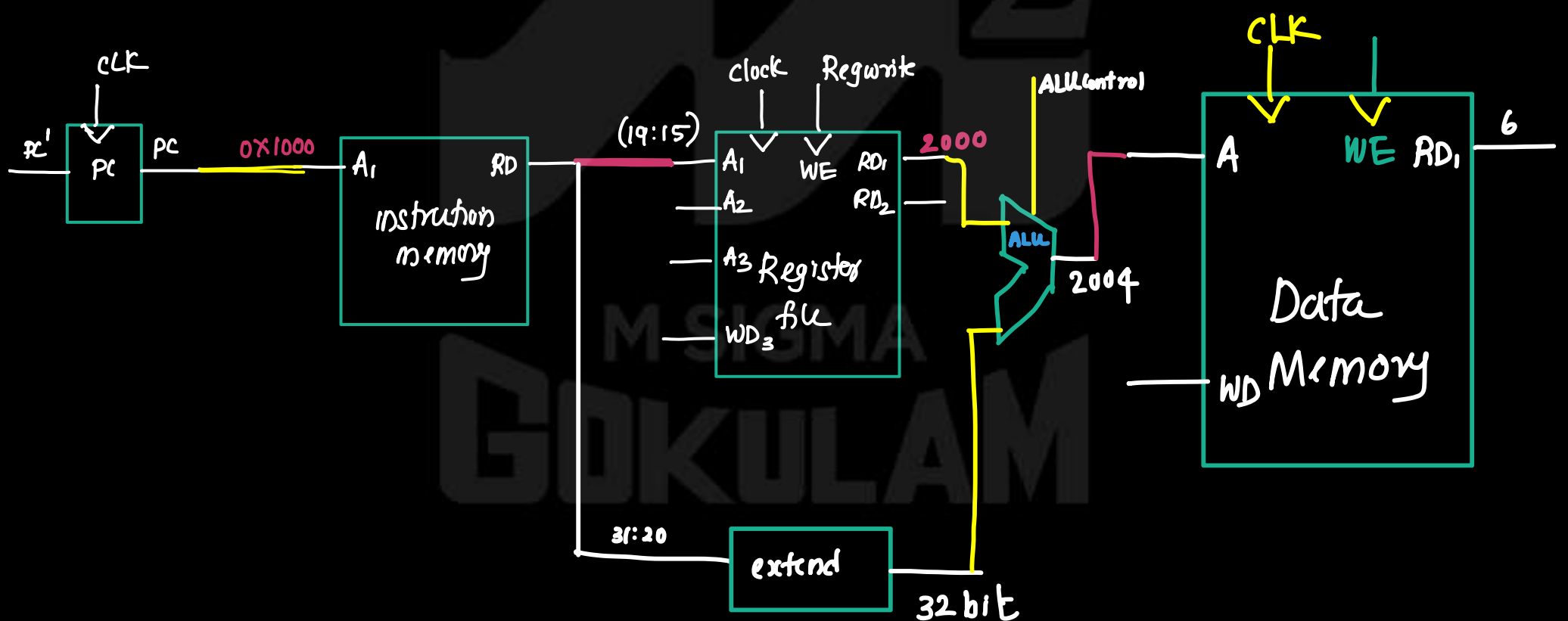
→ In data memory, two operations are possible

① read ② write
for selecting the operation a control signal should be given to data memory. That control signal is called memory write (Memwrite).

this is connected to write enable (WE) pin

If $WE = 1$, then perform write operation
If $WE = 0$, then perform read operation.

→ Data corresponding to address present in the read input is placed to the read data output (RD_1).



→ The data available at RD_1 need to be written in the register (Rs). Instruction bits 7 to 11 ($instn\ 11:7$) represent the destination register. So these bits ($instn\ 11:7$) are connected to A_3 of Register file and RD of Data memory. Connected to WD_3 of Register file.

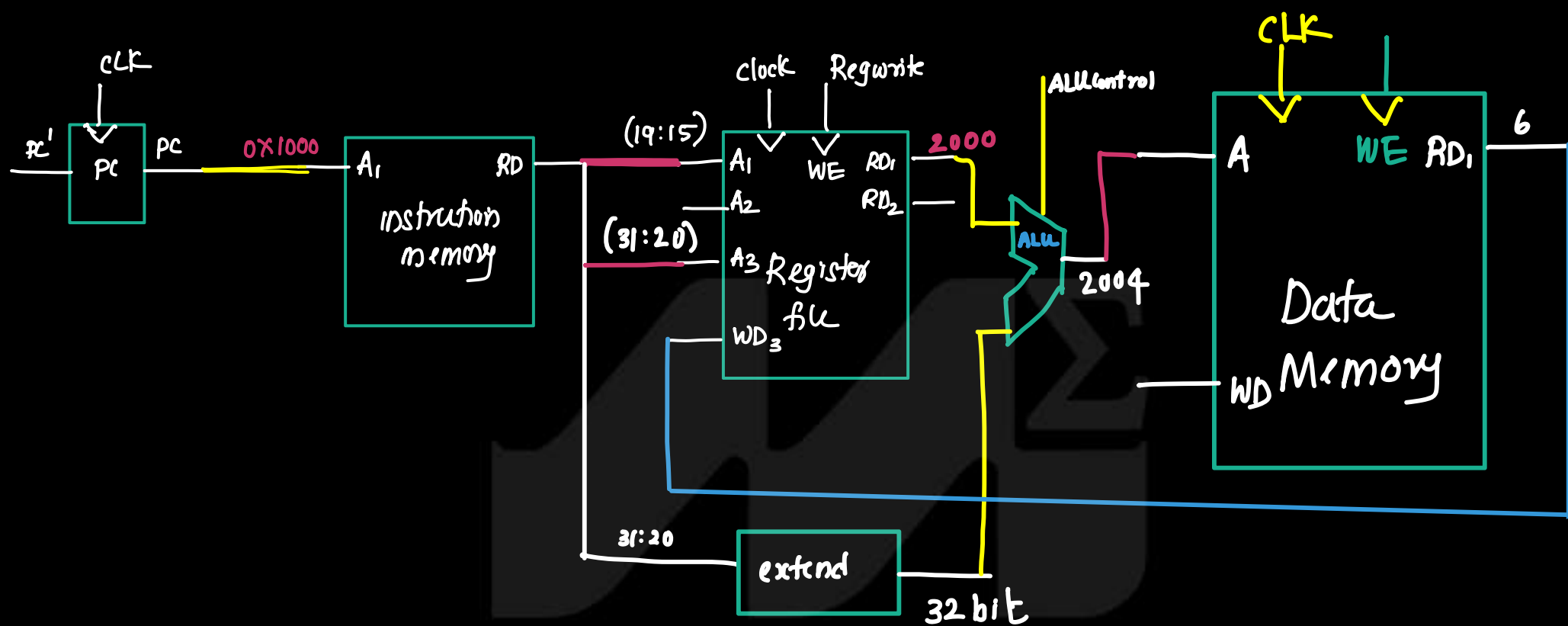
→ Two operations can be performed on register files (Read & write). Operations can be selected by giving a control signal ($Regwrite$).

→ Register write.

if $Regwrite = 1 \rightarrow$ write operation
 if $Regwrite = 0 \rightarrow$ read operation.

(N)

- $\rightarrow$ If control signal ($Regwrite$) at WE pin is one then data available at WD_3 is write on the memory location whose address is available at A_3 .
- $\rightarrow$ Finally data is written on the register file.
- $\rightarrow$

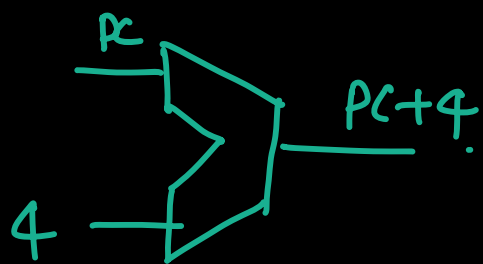


- $\rightarrow$ While executing one instruction, processor must compute the address of next instruction (PC_{next})

$$PC_{next} = PC + 4$$

Instructions are 32 bit (4 byte). so address of the next instruction is $PC + 4$.

- $\rightarrow$ Use an adder to find PC_{next} and result is written on the 'pc'.



Single cycle RISC processor Datapath - Example-2 ^(N)

① Instruction: $sw\ x_5, 4(x_9)$

- store the content of memory location to the register (x_5).

$$\text{effective address} = \text{base address} + \text{offset}$$

$$\text{effective address} = \text{Content of } x_9 + 4$$

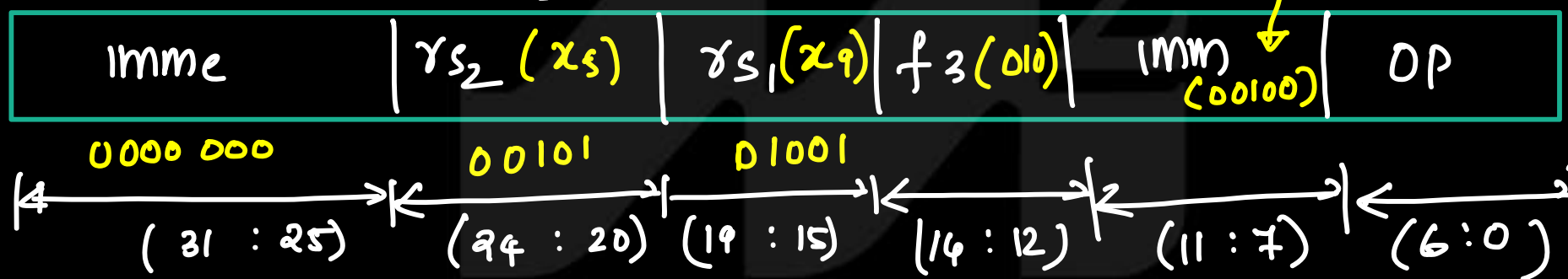
- Content of $x_9 = 2000$

- Content of memory location $2004 = 06$

- $PC = 1000$

'sw' is 's' type instruction

$sw\ x_5, 4(x_9)$



In this instruction immediate value represented by (instn 11:7, instn 31:25).

- ① PC place the address to address input^(A₁) of instruction memory.
- ② instruction memory fetch the instruction and place the instruction in read data output (RD).
- ③ Address calculation:
 - To calculate the effective address, need to read the content of register (x_9) and extend the immedia. value represented by instn 11:7, instn 31:25
 - To read the content of register (x_9), instruction bits (instn 19:15) is connected to address input of register file

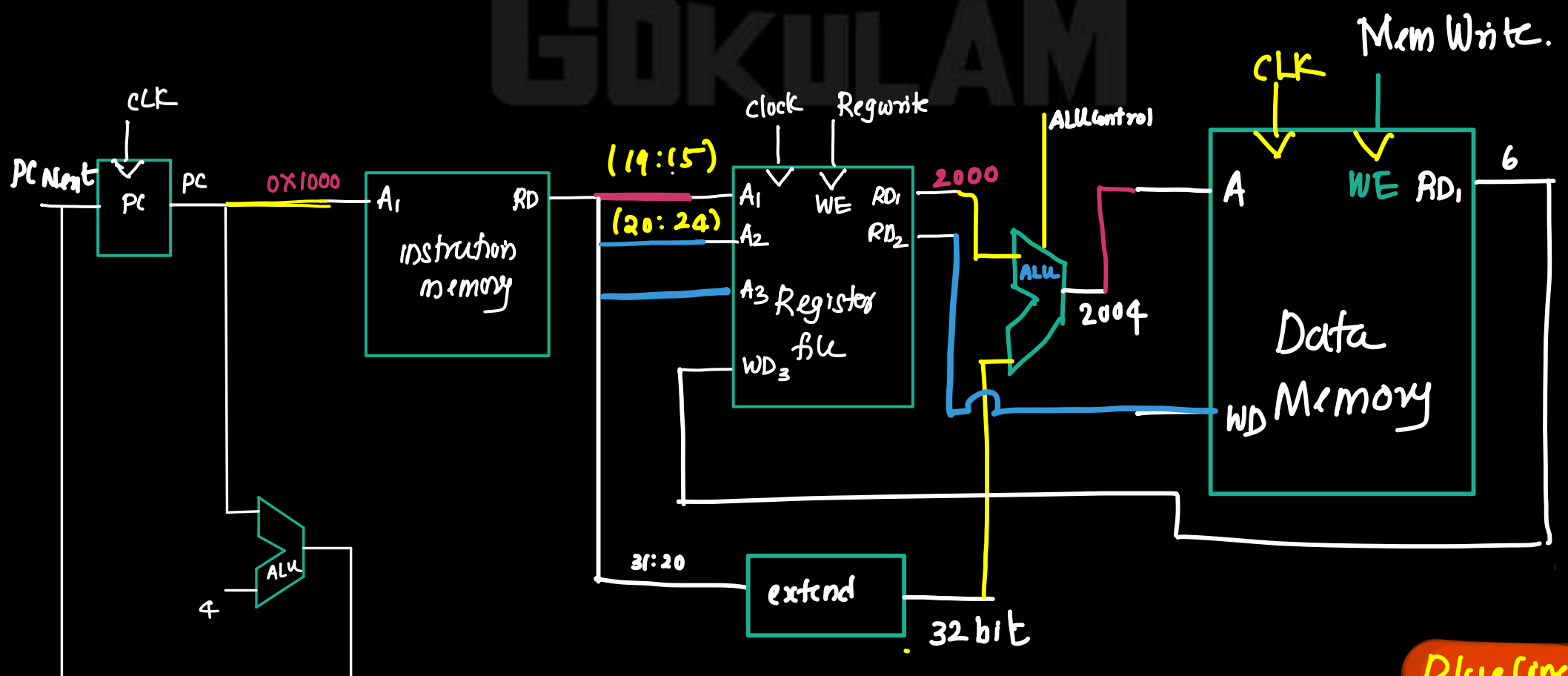
→ register file read the register(x_9) and place the content to read data output (RD1). (N)

→ immediate value represented in the instruction bits (instn 11:7, instn 31:25) is given to extender. and apply control signal (Imm Src = 1). then extender produce 32 bit immediate value.

— base address at the RD1 and 32 bit immediate value given to the ALU. ALU add these values and produce effective address. output of ALU is connected to the address input of data memory.

— to read the content of x_5 , connect instruction bit (instn 20:24) to the address input (A2) of register file. Register file read the content and place in RD2. Connect this RD2 to the WD pin of Data memory.

→ When control signal (Mem Write) is enabled (WE = 1), then data at the WD3 pin is stored at memory location whose address is specified in the address input (A3).



Blue lines
are New
line

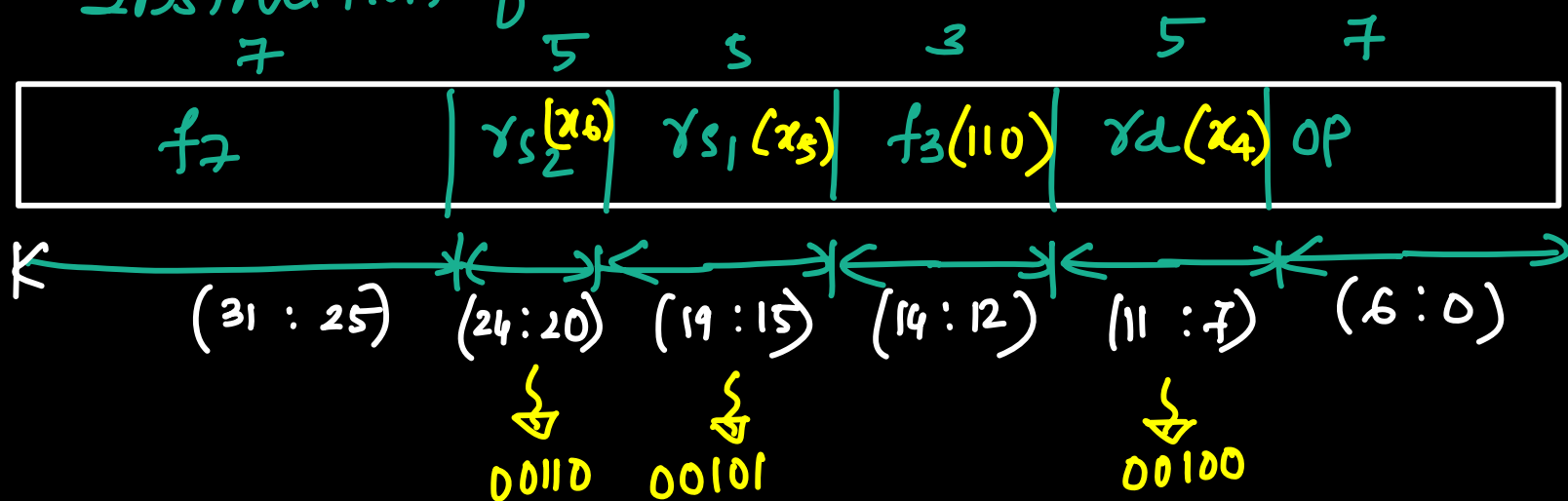
Example - 3

(N)

OR x_4, x_5, x_6

operation — perform OR operation b/w x_5, x_6 and
store the result in Register x_4
→ it is a R-type instruction

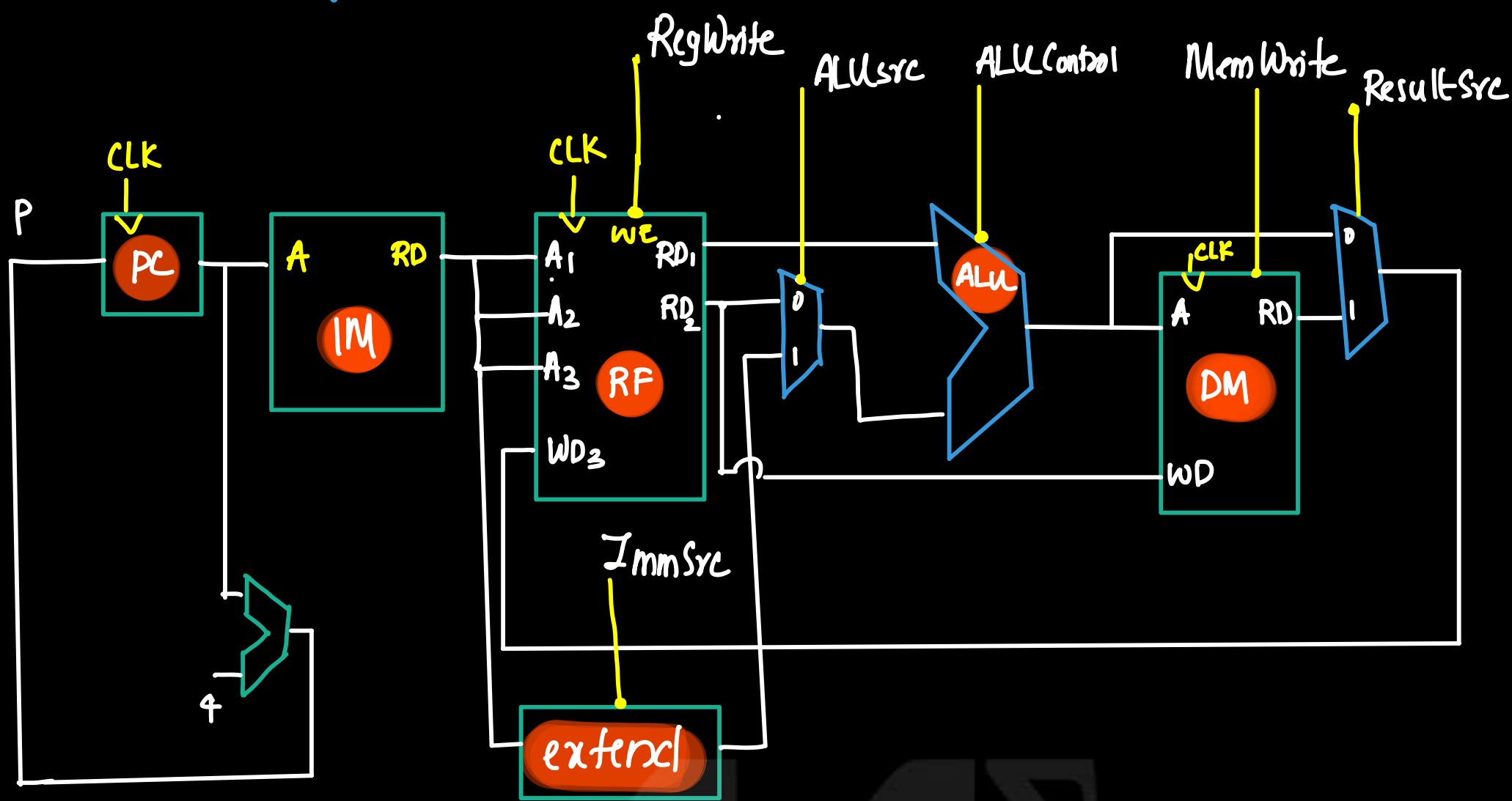
Instruction format



M SIGMA
GOKULAM

Data path

(N)

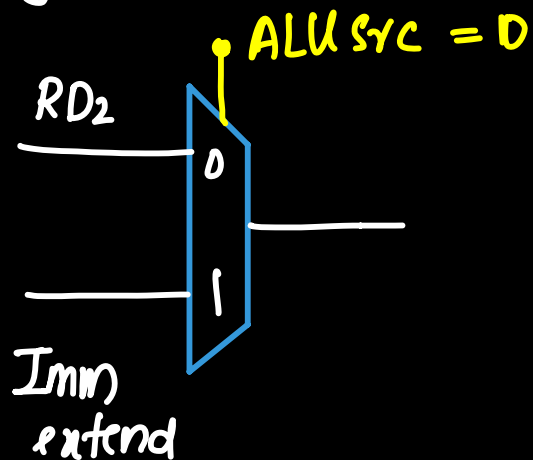


- ① PC Contains address of next instruction to be executed
- ② PC places address to the address input (A) of instruction memory.
- ③ instruction memory fetch the instruction and place in read data output (RD)
- ④ Instruction or x_4, x_5, x_6 .
- ⑤ instruction bits representing x_5 (instn 19:15) connected to address input A_1 of register files.
- ⑥ instruction bits representing x_6 (instn 26:20) connected to A_2 .
- ⑦ Register file read the content of x_5 and place the content in RD_1
- ⑧ Register file read the content of x_6 and place the content RD_2 .
- ⑨ Data available at RD_1 is directly given to the one input of ALU
- ⑩ Data available at RD_2 and output of extender is connected to a multiplexer with a select line

(ALUSrc).

If $ALUSrc = 0$, then multiplier select register file output
If $ALUSrc = 1$, then select Extended immediate value.

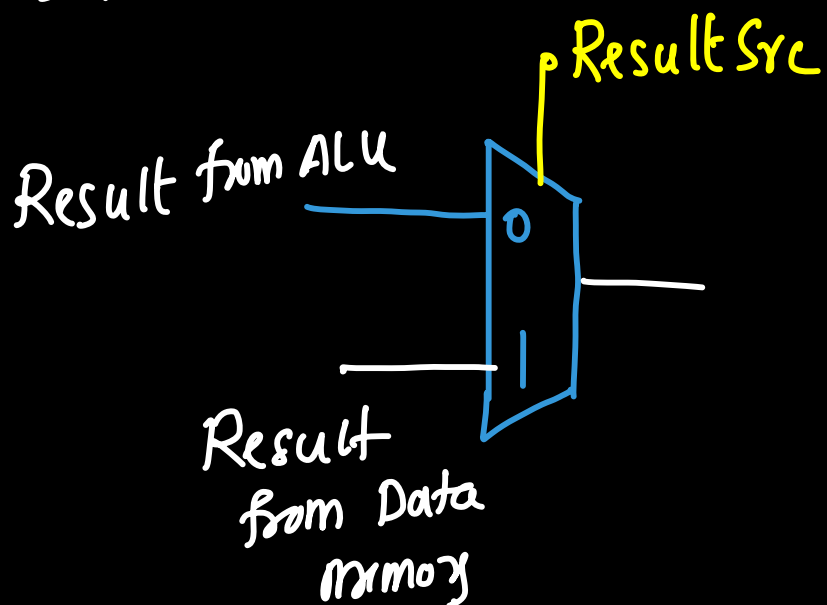
Here register file output (RD_2) is required so $ALUSrc = 0$



⑪ ALU perform the operation b/w register file outputs RD_1 & RD_2 . Operation of ALU is decided by the ALU control signal.

ALU control signal for or operation is 011

⑫ Result of ALU write back to register x_4 . But for Lw/sw instructions result comes from data memory. For Rtype (eg: or s_4, s_5, s_6) result comes from ALU. So we use a multiplex to choose the proper result based on the type of instruction.



If $ResultSrc = 0$, Result selected from ALU

If $ResultSrc = 1$, Result selected from Data memory.

In this example, we need result of ALU so $ResultSrc = 0$

⑬ write back the ALU result in register file (x_4)
(Connect instruction bits corresponding to x_4 to A_3 and make the control signal (RegWrite) to 1)

↳ store the result in register x4.

(N)

Sample Questions – Single cycle Data path.

- ① Explain data path operation for the following instruction in a single cycle RISC processor with the help of a neat diagram.
- ① lw x4, 5(x9)
 - ② sw x5, 5(x9)
 - ③ add s4, s5, s6
 - ④ and s4, s5, s6
 - ⑤ beq, s0, s1, L1

M SIGMA
GOKULAM

Show the state elements of a single-cycle RISC-V processor, and show the datapath of the machine instruction 0062E233, which corresponds to OR instruction.



Pipelined Processor. (Note)

— In pipelined processor, instruction execution process is divided into five stages. Each stage completed in one clock cycle. so that multiple instruction can be executed simultaneously.

— Stages of pipelined processor

- ① Fetch ② Decode ③ execute ④ memory access
⑤ writeback

① Fetch (instruction fetch)

processor reads (fetch) the instruction from the memory.

② Decode (instruction decode)

the instruction is decoded, control signals are generated and read the operands from the register file

eg: add S_1, S_2, S_3

① decode the instruction and by using instruction bits (op, funct3, funct7) decode the instruction

② Read the content of Register file (content of S_2 & S_3).

③ Execution (Ex)

ALU perform the computation such as arithmetic and logical operations.

④ Memory access (memory)

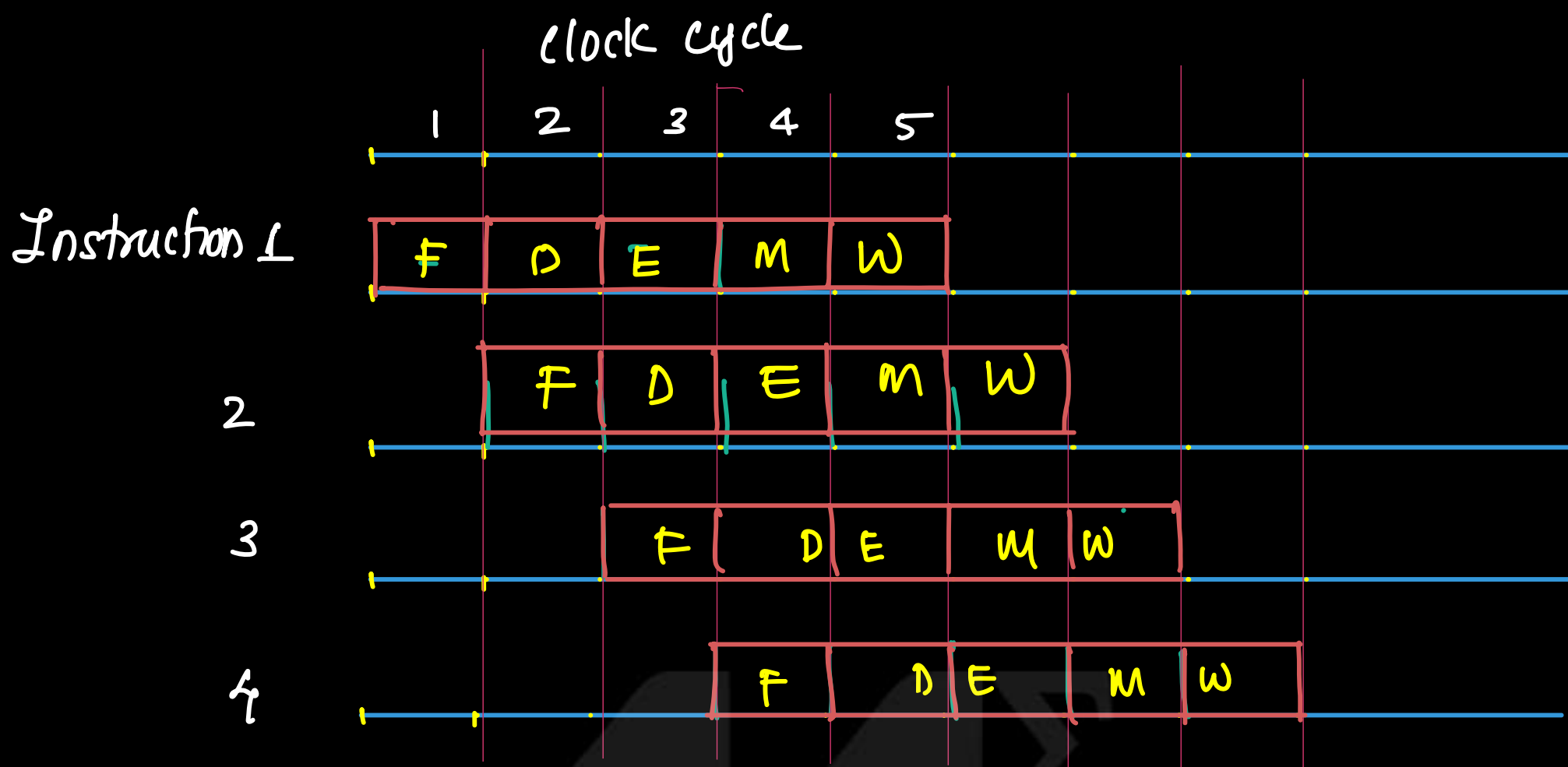
Data memory is accessed to read or write data

⑤ Write back

Final result written to the register file, if needed.

Timing diagram of pipelined process (Performance Comparison of single cycle and pipelined processor)

— Consider 4 instructions (I_1, I_2, I_3, I_4)



- In First cycle, I_1 is fetched from the memory
- In 2nd clock cycle, I_1 enter to the decode stage and I_2 is fetched
- In 3rd clock cycle, I_1 executes, I_2 enter to decode stage and I_3 is fetched
- In 4th clock cycle, I_1 enter to the memory access stage, I_2 executes, I_3 enter to the decode stage, I_4 is fetched
- In 5th clock cycle
 - I_1 → write back the result in Register file
 - I_2 → access the data memory
 - I_3 → execute
 - I_4 → Decode
- at the end of 5th clock cycle First instruction executed
- at the end of 6th clock cycle 2nd instruction executed

→ after 5th clock cycle each instruction is completed in each clock cycle.

i.e. 1 instruction in 1 clock cycle

$$\text{clock cycle time} = 200\text{ps} = 200 \times 10^{-12} \text{ s}$$

$$1 \text{ instruction} \longrightarrow 200 \times 10^{-12} \text{ s}$$

$$\times \longleftarrow 1 \text{ s}$$

$$x = \frac{1}{200 \times 10^{-12}} = 5000 \times 10^6 = 5 \times 10^9 = 5 \text{ billion}$$

i.e. In this pipelined processor can execute 5 billion instruction in one second.

i.e. throughput = 5 billion instruction per second.

Total time taken for the single cycle process to completing one instruction = time for Fetch + time for decode + Time for execution + Time for memory access + Time for writeback

$$T = 200\text{ps} + 100\text{ps} + 120 + 200 + 60 = 680\text{ps}$$

one instruction in 680×10^{-12} second

$$1 \text{ instruction} \longrightarrow 680 \times 10^{-12} \text{ s.}$$

$$\times \longleftarrow 1 \text{ s}$$

$$x = \frac{1}{680 \times 10^{-12}} = 1470 \times 10^6 = 1.47 \times 10^9 = 1.47 \text{ billion}$$

For single cycle processor can complete 1.47 billion instruction in one second

throughput = 1.47 billion instruction per second
Single

through put of pipelined processor = 5 billion instruction per second.

So through put of pipelined processor = 3 times of through put of single cycle processor.



Module 2

N

A non-pipelined RISC-V processor has a cycle time of 800 ps. When converted to a 5-stage pipeline, the longest stage delay is 200 ps. Due to pipeline registers, an additional 20 ps overhead is added to each stage. Calculate the speedup achieved by pipelining for a large number of instructions, assuming no stalls.

non pipelined processor cycle time = 800 ps
(processor executes one instruction in one clock cycle)

$$1 \text{ instruction} \longrightarrow 800 \times 10^{-12} \text{ s}$$
$$\chi \longleftarrow 1 \text{ s}$$

$$\chi_{\text{non}} = \frac{1}{800 \times 10^{-12}} = \underline{\underline{1.25 \times 10^9}}$$

$\chi \rightarrow$ no. of instructions executed in 1 sec $\rightarrow$ throughput

$$\text{throughput of non pipelined} = \chi_{\text{non-pipelined}} = \frac{1}{800 \times 10^{-12}}$$

Pipelined sm:

cycle time = 200 ps

Additional delay = 20 ps

total time = 220 ps

pipelined sm complete one instruction in every clock cycle (After 5th clock cycle)

$$1 \text{ instruction} \longrightarrow 1 \text{ clock cycle time (220 ps)}$$
$$\chi \longleftarrow 1 \text{ s}$$

$\chi \rightarrow$ throughput of pipelined sm

$$\chi_{\text{pip}} = \frac{1}{220 \times 10^{-12}} = \frac{10^{12}}{220} = \underline{\underline{4.54 \times 10^9}}$$

$$\chi_{\text{non}} = 1.25 \times 10^9$$

$$\chi_{\text{pip}} = 4.54 \times 10^9$$

$$\frac{\chi_{\text{pip}}}{\chi_{\text{non}}} = \frac{4.54 \times 10^9}{1.25 \times 10^9} = \underline{\underline{3.63}}$$

$$\chi_{rip} = 3.63 \chi_{nop}$$

(N)



Operation of Pipelined processor (Note)

→ In pipelined processor, instruction execution process is divided in 5 stages.

→ each pipe line stage is represented with its major components

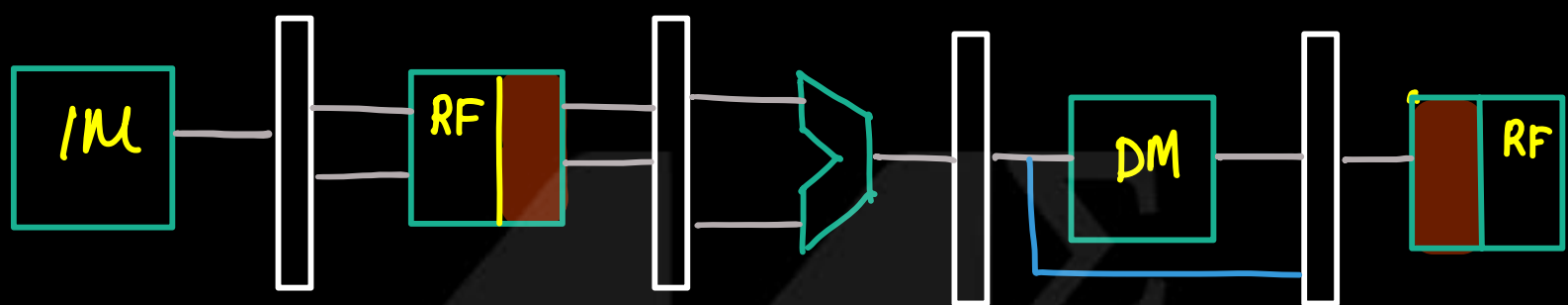
Fetch — Instruction Memory (IM)

Decode — Register file (RF)

Execution — ALU

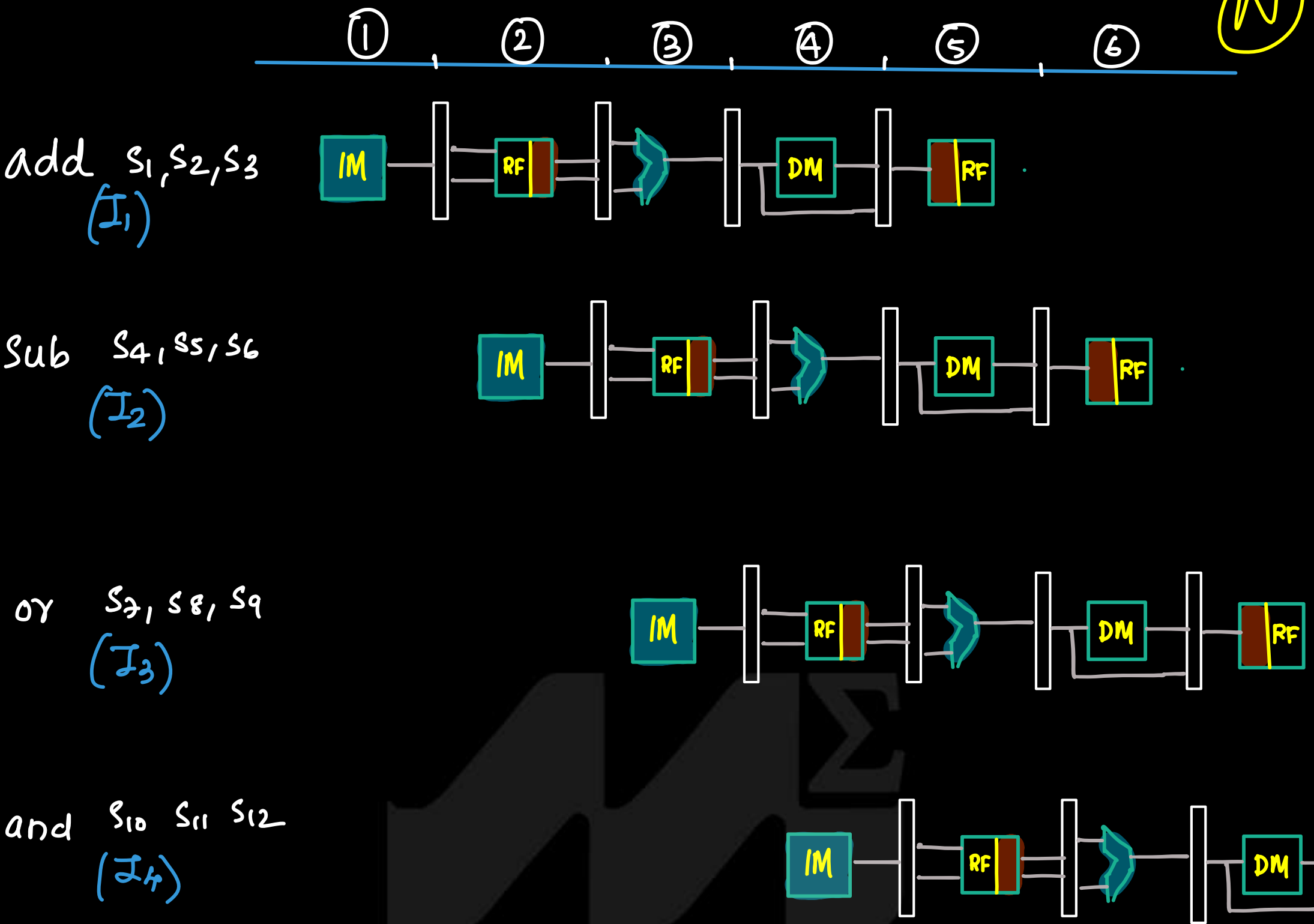
Memory — Data Memory (DM)

Write Back — Register file



M SIGMA
GOKULAM

(N)



— In first clock cycle, I_1 is fetched from the IM.

→ In 2nd clock cycle
 I_1 reads s_2, s_3 in the second half cycle of cycle 2
 I_2 fetched from the memory

→ In 3rd clock cycle
 I_1 → execute and result will be available at ALU output
 I_2 — reads s_5, s_6
 I_3 — fetch the instruction

→ In 4th clock cycle
 I_1 → Since this instruction is not a load/store instruction. So Data memory is idle (not used.)

$I_2 \rightarrow$ execute subtraction

$I_3 \rightarrow$ Reads s_8, s_9

(N)

$\rightarrow$ In 5th clock cycle

$I_1 \rightarrow$ store the result in register file in the first half of cycle 5. after first half of cycle 5 result will be available in register s_1

$I_2 \rightarrow$ not accessing DM

$I_3 \rightarrow$ execute the instruction

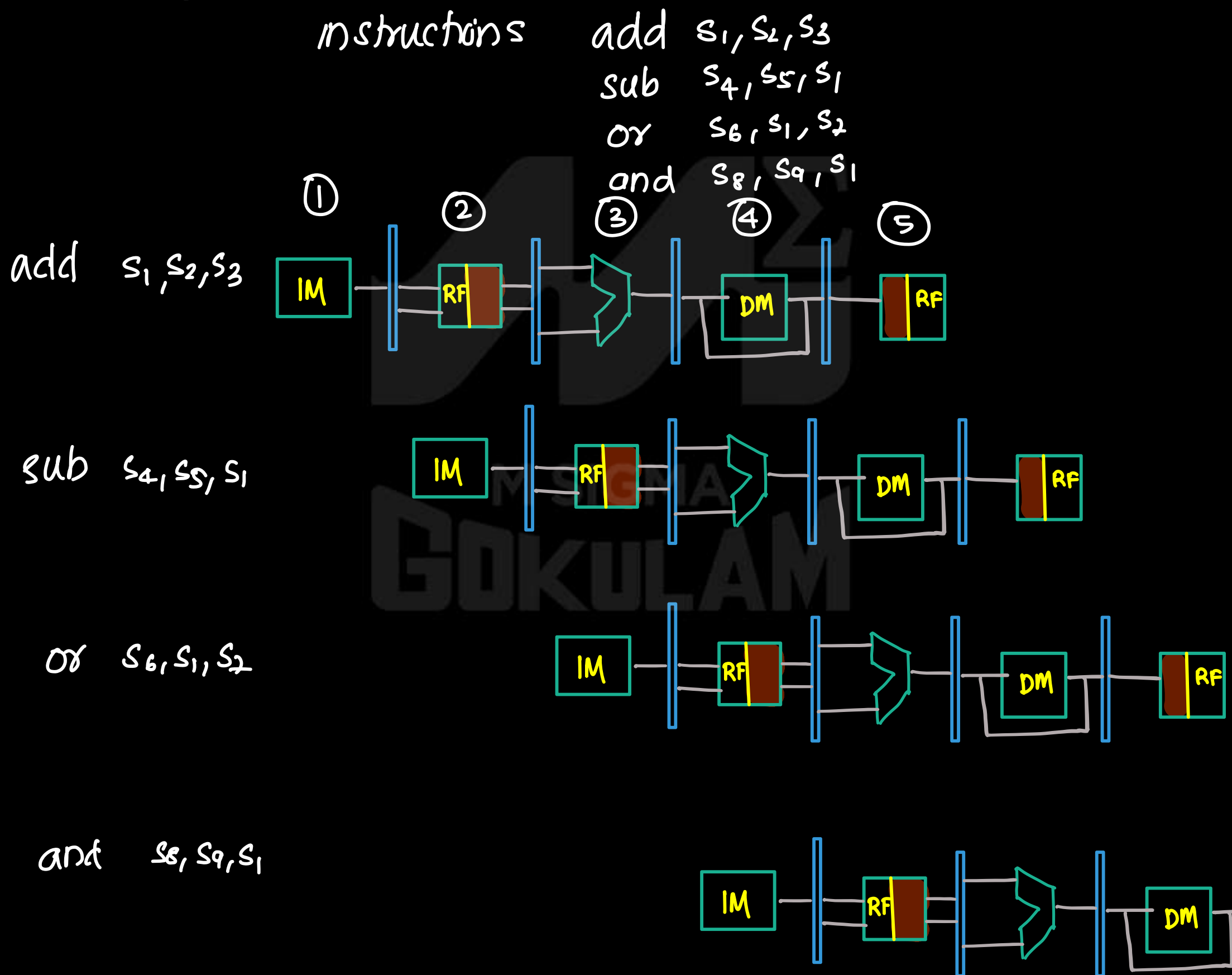
Execution of First instruction completed at the end of cycle 5. (i.e. sum is available in Register s_1 after the end of first half of cycle 5).



Pipeline hazards. (Note)

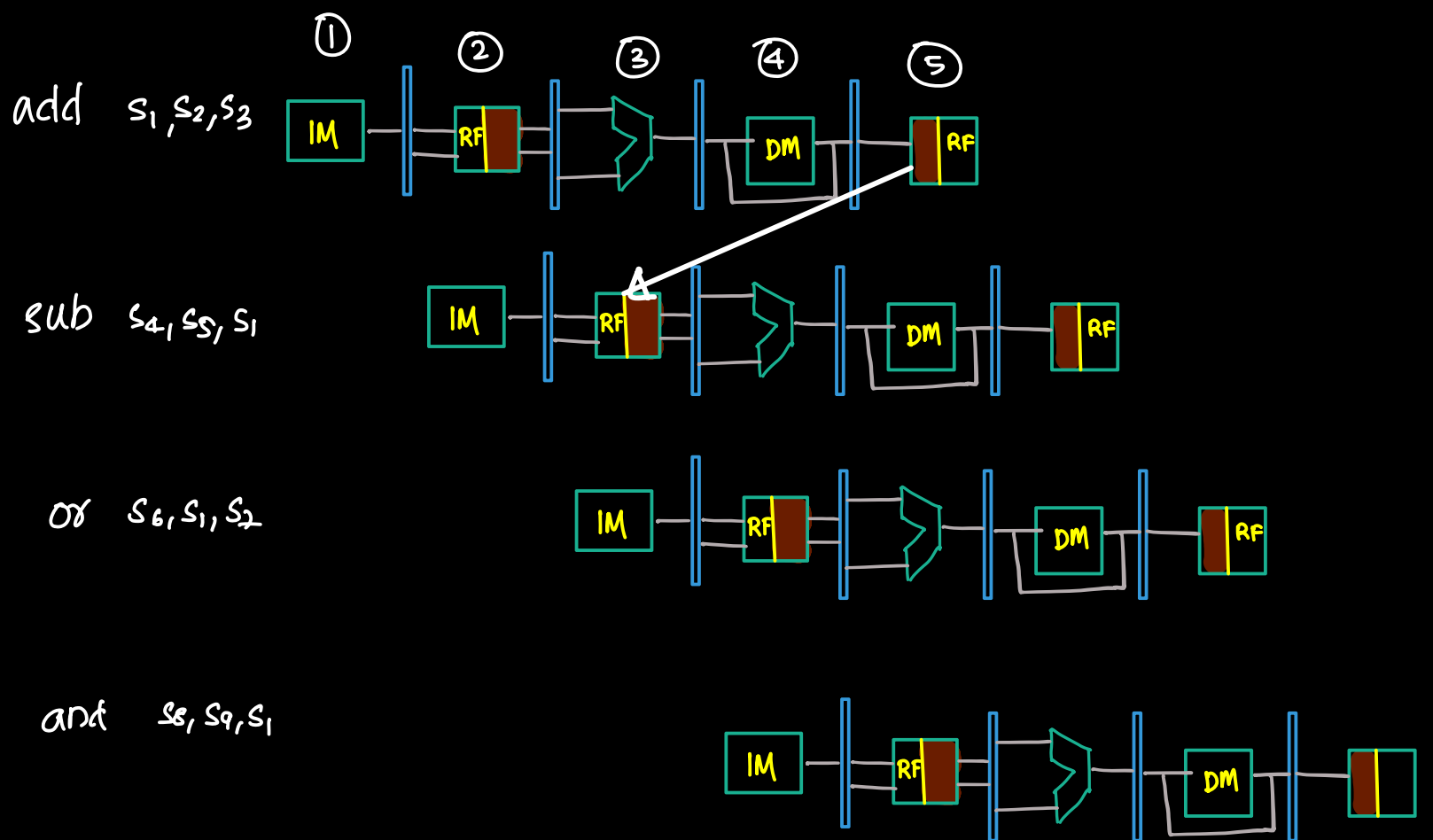
— In a pipelined processor, instruction execution is divided into different stages. and multiple instructions are executed simultaneously by overlapping different stages. This introduces a problem called pipeline hazards.

→ Consider a 4 instructions executed simultaneously using 5-stage pipelining.



① Add instruction store the sum in cycle 5., i.e. result of 1st instruction available at end of 5th clock cycle. But 2nd instruction (sub) reads s_1 in cycle 3. Register s_1 contains the result of add instruction only at the end of 5th clock cycle. So and instruction (sub) reads wrong.

value of s_i . this create a problem. this is called hazards. (N)



similarly 3rd instruction (or) reads the content of s_1 in cycle 4. But correct value is not available. this will cause hazard.

Here one instruction that depends on the result of another instruction that has not yet completed.

pipe line hazards

When one instruction is dependent on the result of another that has not yet completed, a hazard occurs.

Different types hazards

- ① Data hazard
- ② Control hazard.

① Data hazard

②

Data hazard occurs in a pipelined processor when an instruction depends on the result of a previous instruction that has not yet completed execution. To ensure correct program execution, data hazard must be detected and resolved using different methods.

Methods to solve data hazards

- ① solving data hazards with forwarding
- ② solving data hazards with stalls

① solving data hazards with forwarding

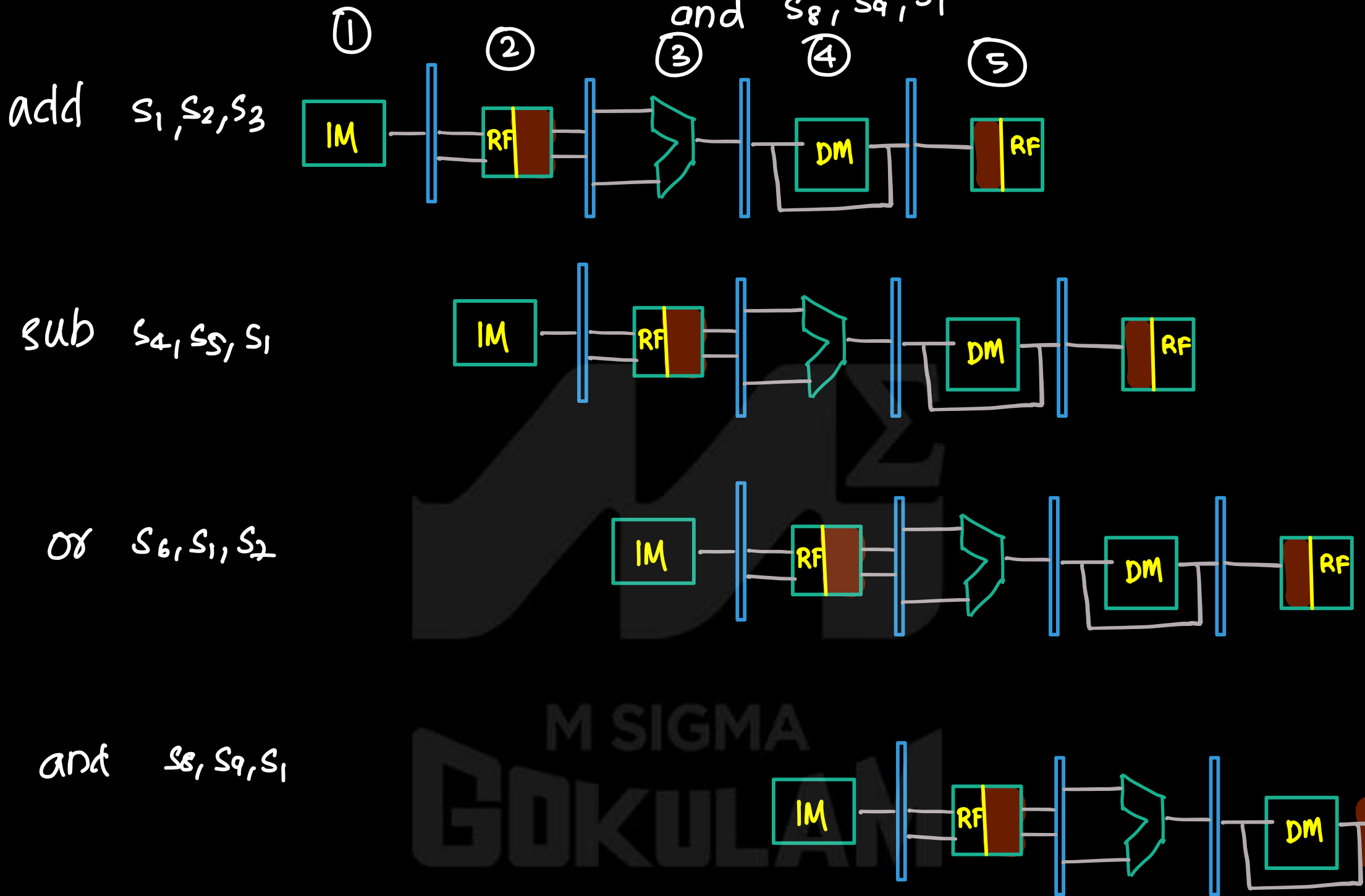
— Forwarding is a technique in which result of an instruction is sent directly from a later pipeline stage to an earlier stage without waiting for it to be written back to the register.

— Forwarding a result from memory or write back stage to a dependent instruction in the execute stage.

M SIGMA
GOKULAM

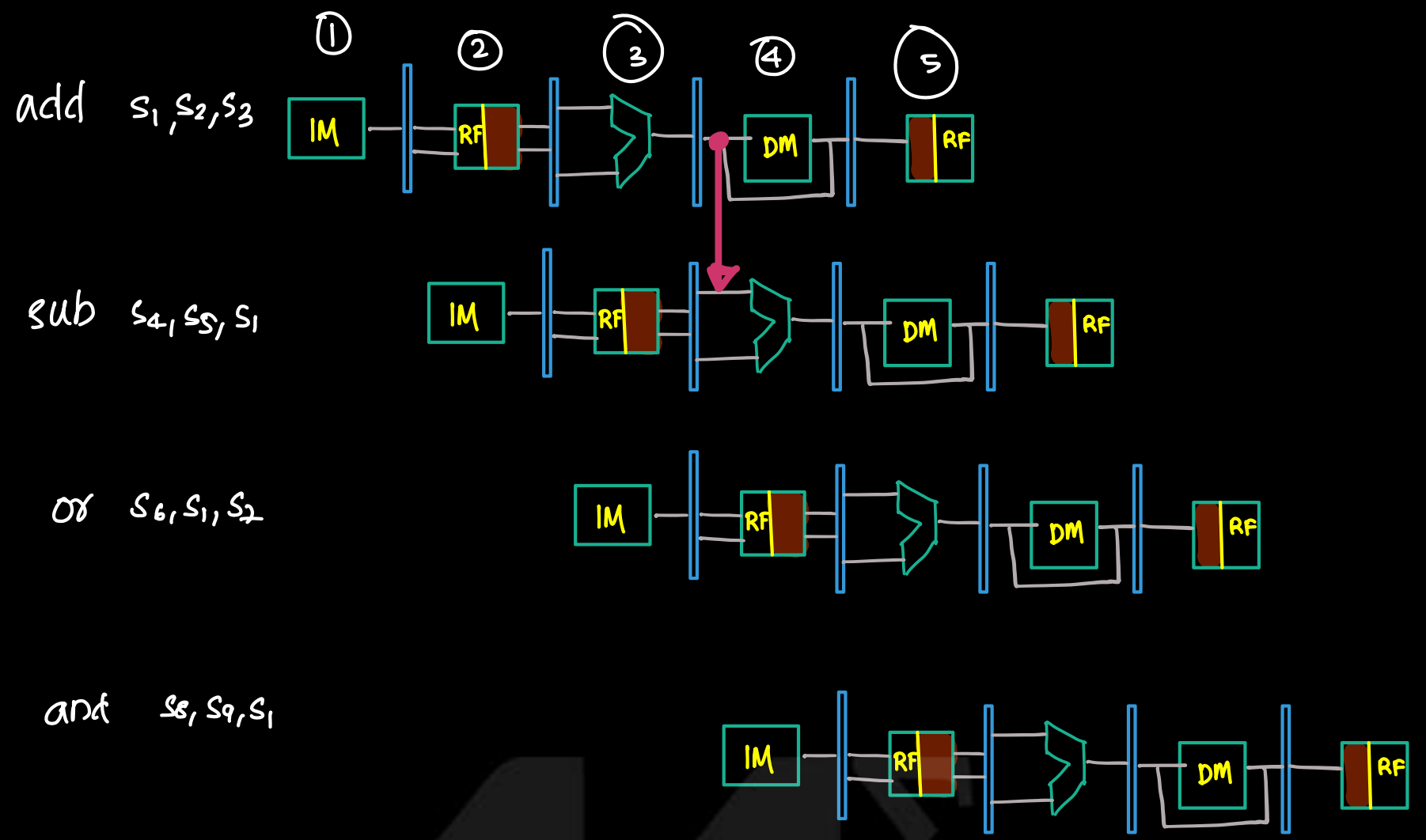
→ Consider a 4 instructions executed simultaneously using 5-stage pipelining. (Q)

Instructions add s_1, s_2, s_3
 sub s_4, s_5, s_1
 or s_6, s_1, s_2
 and s_8, s_9, s_1



① Add instruction store the sum in cycle 5, i.e. result of 1st instruction available at end of 5th clock cycle. But 2nd instruction (sub) reads s_1 in cycle 3. Register s_1 contains the result of add instruction only at the end of 5th clock cycle. So 2nd instruction (sub) reads wrong value of s_1 . This creates a problem. This can be solved by forwarding.

→ In cycle 4, result of 1st instruction forwarded from memory stage to execution state of sub instruction.



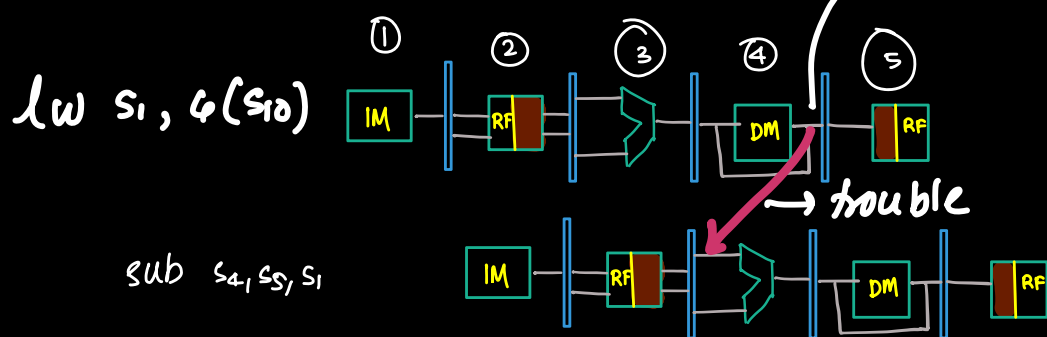
② solving data hazards with stalls

(N)

consider a load instruction

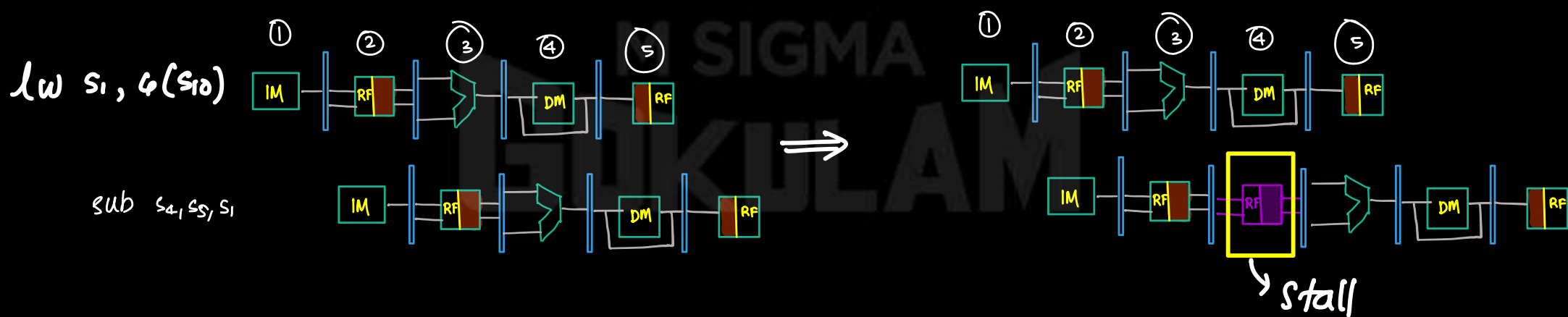
eg: $lw\ s_1, 4(s_{10})$

- load the content of memory location to the register s_1 .
- this instruction finish reading data at the end of memory stage.



Content of memory location will be available at the end of memory stage. But sub instruction need the content of s_1 in the beginning of 4th clock cycle. so data cannot be forward to execute state of sub instruction.

This problem can be solved by holding up operation until the data is available.



→ Now Result of 'lw' instruction available at the end of memory stage (4th cycle). and sub instruction needs result of 'lw' in 5th clock cycle.

2

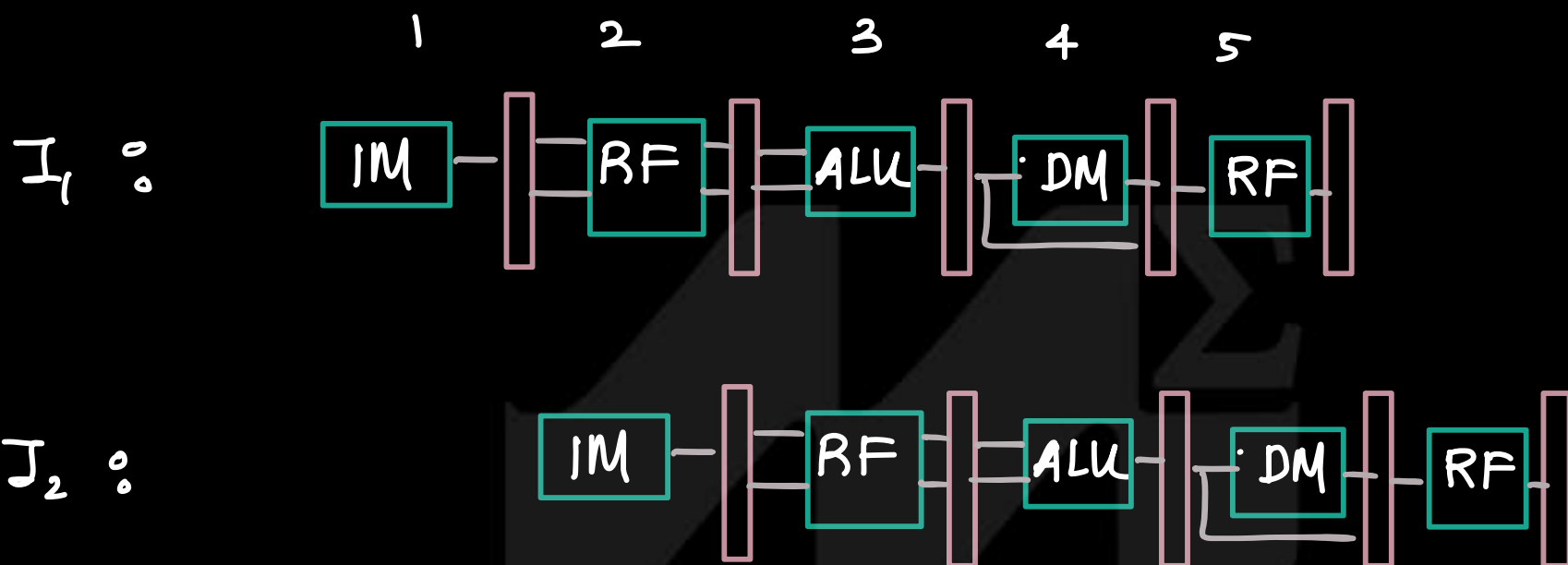
Identify the hazard that occurs when executing the following instructions in sequence in a pipelined RISC-V machine. Propose any one solution to overcome the issue.

lw X₁₀, 0(X₁₁)

add X₁₂, X₁₀, X₁₃

two instruction - I_1 - lw x₁₀, 0(x₁₁)
 I_2 - add x₁₂, x₁₀, x₁₃

consider a 5 stage pipelined sm.



→ Instruction '1' [lw x₁₀, 0(x₁₁)]

→ load the content of memory location pointed by sum of content of register(x₁₁) and immediate value
 → Content of x₁₀ will be available at the end of 4th clock cycle / beginning of 5th clock cycle.

→ But for 2nd instruction read the value of x₁₀ in the 3rd clock cycle. but actual value of x₁₀ will be available at the beginning of 5th clock cycle. so it create a hazard.

I_1 - lw x₁₀, 0(x₁₁) → x₁₀ → write the value to ①
 I_2 - add x₁₂, x₁₀, x₁₃ → x₁₀ → Read the value of x₁₀

② & ①
 read after write (RAW)

the hazard can be overcome using pipeline stall.

add $x_5, x_1, x_2 \rightarrow x_1 \Rightarrow \text{Read } ①$
add $x_1, x_3, x_4 \quad x_1 \Rightarrow \text{write } ②$

② $\Leftarrow$ ①
write after Read



Control Hazards.

(2)

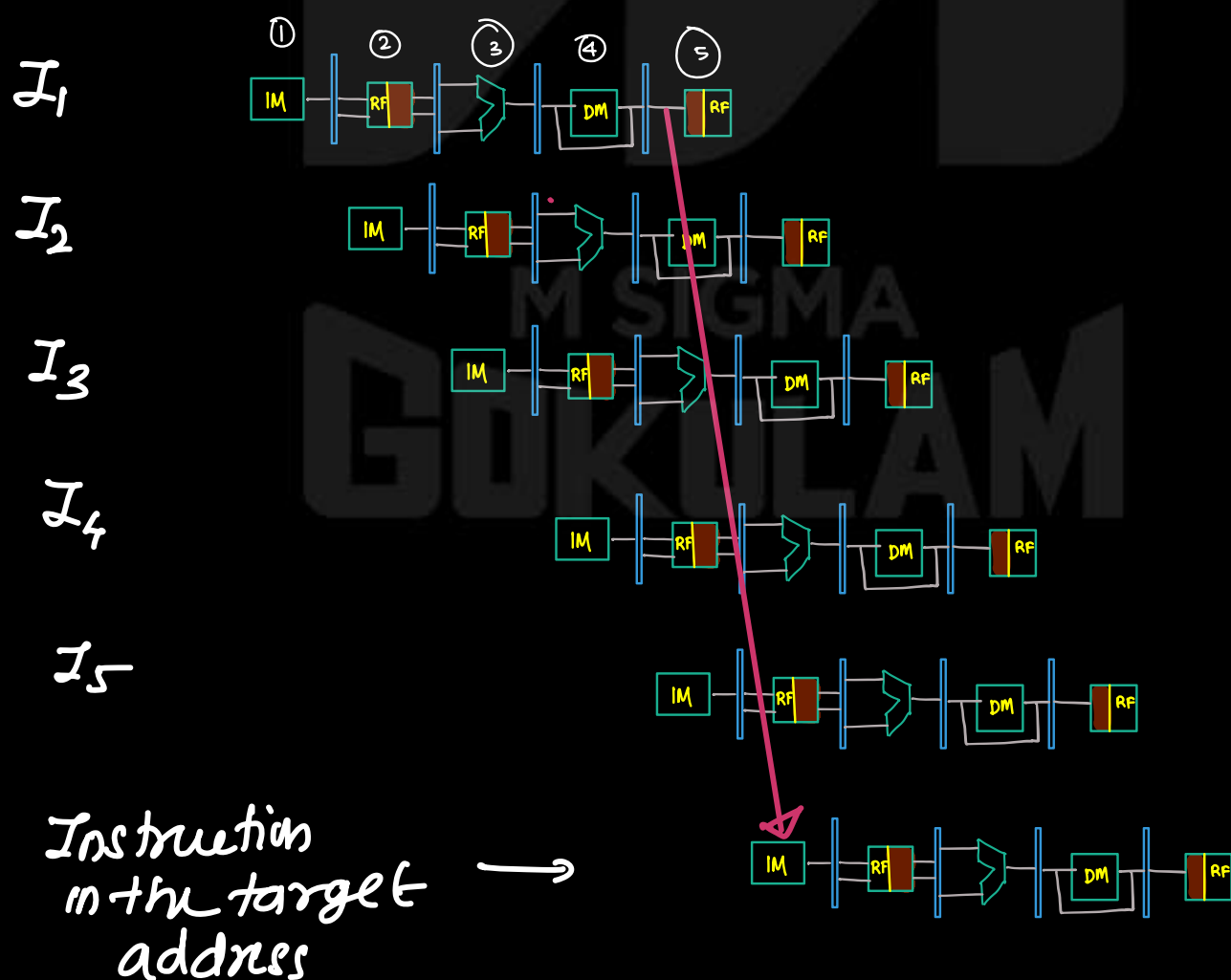
- In pipelined processor, control hazards occurs due to the branch instructions. Because processor does not know what instruction to fetch next, branch target address is not calculated at time of next instruction is fetched

→ Consider a 5 instructions I_1, I_2, I_3, I_4, I_5 , in which I_1 is the branch instruction.

Eg: $I_1 \rightarrow \text{beq } s_1, s_2, L_1$
branch if content of s_1 & s_2 are equal
 $L_1 \rightarrow$ target address.

- $I_1 \rightarrow$ Branch instruction
 $I_2, I_3, I_4, I_5 \rightarrow$ sequential instruction.

Pipe line execution



- Assume that branch target address is available at the beginning of the 5th clock cycle

— clock wise progress

(N)

Cycle-1 : I_1 is fetched

Cycle-2 : I_1 decoded, I_2 Fetched

Cycle-3 : I_1 executed, I_2 Decoded, I_3 Fetched

Cycle-4 : I_1 Memory stage, I_2 executed, I_3 Decoded

Cycle-5 : I_1 write back stage (Branch target address)

By the beginning of cycle 5, the processor has already fetched I_2, I_3, I_4, I_5 (4 instructions)

→ When branch is taken, then processor execute the instruction in the target address and flushed the already fetched wrong path instruction (I_2, I_3, I_4, I_5).

→ So mispenalty = 4 cycle (Because 4 instruction must be flushed.)

How to reduce this mispenalty.

— To reduce the number of flushed instructions, we try to predict branch target earlier

— If the branch target available in cycle 3 (execution stage of I_1). then fewer instructions fetched wrongly. Hence flush count reduces.

